

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC PHENIKAA**



BÁO CÁO TIỂU LUẬN
LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG
(CSE703029-1-3-25(N01))
HỆ THỐNG QUẢN LÝ THƯ VIỆN

SINH VIÊN THỰC HIỆN:

1. Nguyễn Mạnh Toàn (23010539)
2. Nguyễn Văn Lộc (23013697)
3. Trần Công Thành Đạt (23010782)
4. Nguyễn Mạnh Đạt (23010528)
5. Trần Thị Thu Hoài (22010720)

GIẢNG VIÊN HƯỚNG DẪN: TS. Trần Đình Nam Sơn

HÀ NỘI, 7/7/2026

MỤC LỤC

Bảng phân chia công việc	v
LỜI MỞ ĐẦU	1
1 TỔNG QUAN VỀ BÀI TOÁN VÀ ĐỀ TÀI ĐẶT RA	3
1.1 Lý do chọn đề tài	3
1.1.1 Thực trạng quản lý lưu thông sách tại các thư viện quy mô vừa và nhỏ	3
1.1.2 Nhu cầu tối ưu hóa tốc độ xử lý dữ liệu và trải nghiệm quản trị thời gian thực	3
1.1.3 Thách thức áp dụng nguyên lý OOP vào giải quyết bài toán thực tế	4
1.2 Mục tiêu của dự án	4
1.3 Phạm vi áp dụng và đối tượng nghiên cứu	5
1.3.1 Đối tượng nghiên cứu	5
1.3.2 Phạm vi áp dụng	5
2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	6
2.1 Bốn nguyên lý cốt lõi của Lập trình hướng đối tượng (OOP)	6
2.1.1 Tính đóng gói (Encapsulation)	6
2.1.2 Tính kế thừa (Inheritance)	6
2.1.3 Tính đa hình (Polymorphism)	7
2.1.4 Tính trừu tượng (Abstraction)	7
2.2 Mẫu thiết kế Chiến lược (Strategy Pattern) trong tính toán tiền phạt	7
2.3 Kiến trúc tách biệt Frontend - Backend (Decoupled Architecture)	8
2.3.1 Backend (Java Spring Boot REST API)	8
2.3.2 Frontend (React.js SPA)	9
2.3.3 Cơ chế CORS (Cross-Origin Resource Sharing)	9
2.4 Công nghệ lưu trữ dữ liệu phi quan hệ bằng tệp tin cứng JSON	9

3	PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	11
3.1	Phân tích yêu cầu chức năng (Use Case Analysis)	11
3.1.1	Phân hệ Quản lý thực thể Danh mục (Core CRUD)	11
3.1.2	Phân hệ Điều phối Lưu thông Sách (Transaction Management)	11
3.1.3	Phân hệ Giám sát Luồng Dữ liệu (Real-time Monitoring)	12
3.2	Thiết kế Sơ đồ lớp chi tiết (Detailed Class Diagram Analysis)	12
3.2.1	Tầng Thực thể Nghiệp vụ (Domain Models Layer)	12
3.2.2	Tầng Truy xuất và Lưu trữ Dữ liệu (Persistence / Repositories Layer)	13
3.2.3	Tầng Logic Nghiệp vụ Trung tâm (Business Services Layer)	13
3.2.4	Tầng Đối tượng Chuyển đổi Dữ liệu (Data Transfer Objects - DTO)	14
3.2.5	Tầng Chiến lược Tài chính Đa hình (Polymorphic Strategy Pattern)	14
3.2.6	Tầng Định tuyến API và Bẫy lỗi Tập trung (Controllers & Web Security Boundary)	14
3.3	Thiết kế Luồng xử lý Dữ liệu và Cấu trúc Lưu trữ Tập JSON	15
3.4	Cấu Trúc Thư Mục Dự Án	15
4	TRIỂN KHAI CÀI ĐẶT VÀ MÃ NGUỒN CỐT LÕI	17
4.1	Mã nguồn Tầng Mô hình (Model Layer)	17
4.1.1	Cấu trúc phân cấp kế thừa và Đóng gói thực thể người dùng (User, Reader)	17
4.1.2	Hiện thực hóa đa hình các nhóm độc giả đặc thù (StudentReader, PriorityStudentReader, LecturerReader)	20
4.1.3	Thiết kế cấu trúc phức hợp và Đối tượng bất biến (BorrowTicket, BorrowTicketDetail, Book)	21
4.1.4	Mô hình phẳng hóa dữ liệu giao tiếp mặt trước (BorrowRecord, ReturnRecord)	25
4.2	Mã nguồn Tầng Nghiệp vụ (Service Layer)	27
4.2.1	Logic điều khiển Lưu thông và Quyết toán đa hình (BorrowReturnService.java)	27
4.2.2	Kiểm tra ràng buộc nghiệp vụ và Quản lý danh mục Thực thể (BookService, ReaderService)	28
4.3	Mã nguồn Xử lý Hoàn trả sách và Quyết toán tài chính	31
4.4	Mã nguồn Giao diện Frontend (React.js) - Bảng điều khiển Giám sát	34
4.4.1	Cơ chế quản lý trạng thái và Gọi API bất đồng bộ (App.jsx)	34
4.4.2	Cấu trúc giao diện cấu trúc khối Bảng giám sát Lưu thông Sách	36

5	THỬ NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ	38
5.1	Kịch bản kiểm thử (Test Cases)	38
5.1.1	Kịch bản kiểm thử 1: Xác thực tính toàn vẹn định dạng lưu trữ UTF-8	38
5.1.2	Kịch bản kiểm thử 2: Bẫy chặn logic hạn mức mượn sách tối đa	38
5.1.3	Kịch bản kiểm thử 3: Quyết toán tài chính và Kiểm thử đa hình Strategy Pattern	39
5.2	Ảnh chụp minh chứng giao diện hệ thống (Screenshots Walkthrough)	40
5.2.1	Giao diện Bảng điều khiển tổng quan (Dashboard)	40
5.2.2	Giao diện Quản lý danh mục Kho sách và Độc giả khôi phục trọn vẹn	41
5.2.3	Hộp thoại thông báo tính phí phạt trễ hạn thời gian thực khi trả sách	42
	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	43
	Tài liệu tham khảo	45

DANH SÁCH HÌNH VẼ

2.1	Sơ đồ cấu trúc Strategy Pattern FinePolicy	8
3.1	Giao diện phân hệ Quản lý thông tin và số lượng tồn kho Sách	12
3.2	Cấu trúc ánh xạ tệp tin cứng readers.json phục vụ Deserialize đa hình	15
3.3	Giao diện phân hệ Quản lý thông tin và số lượng tồn kho Sách	16
5.1	Giao diện Bảng điều khiển tổng quan (Dashboard) tích hợp các cổng số liệu thống kê	40
5.2	Giao diện phân hệ Quản lý hồ sơ và phân cấp nhóm Độc giả	40
5.3	Giao diện phân hệ Quản lý thông tin và số lượng tồn kho Sách	41
5.4	Giao diện cửa sổ Giám sát điều phối mượn trả sách thời gian thực	41

BẢNG PHÂN CHIA CÔNG VIỆC

STT	Họ và tên	MSSV	Nhiệm vụ được giao
1	Nguyễn Mạnh Toàn	23010539	Phần 2.1: Quản lý bạn đọc + Báo cáo
2	Nguyễn Văn Lộc	23013697	Phần 2.2: Quản lý sách + Báo cáo
3	Trần Công Thành Đạt	23010782	Phần 2.3: Mượn sách + Báo cáo
4	Nguyễn Mạnh Đạt	23010528	Phần 2.4: Trả sách + Báo cáo
5	Trần Thị Thu Hoài	22010720	Phần 2.5: Tính phí phạt + Báo cáo

LỜI MỞ ĐẦU

Trong kỷ nguyên số hóa hiện nay, cuộc cách mạng công nghiệp lần thứ tư đã và đang tái định hình toàn diện hạ tầng kỹ thuật và phương thức vận hành của các cơ sở giáo dục đại học. Trong đó, hệ thống thư viện không còn đơn thuần là kho lưu trữ tài liệu vật lý, mà đã chuyển mình thành các trung tâm tích hợp dữ liệu, phục vụ nhu cầu tra cứu, nghiên cứu và học tập thời gian thực của giảng viên và sinh viên. Theo các nghiên cứu kinh điển về kỹ nghệ phần mềm của Sommerville [1] và Pressman [2], để đáp ứng khối lượng giao dịch lưu thông ngày một lớn, các hệ thống quản trị thông tin hiện đại đòi hỏi tính chính xác tuyệt đối, tốc độ tương tác cao, khả năng chịu tải tốt và luồng lưu chuyển dữ liệu phải đảm bảo tính thông suốt, liên tục.

Tuy nhiên, qua khảo sát thực tế tại các thư viện phân khoa hoặc các trung tâm thông tin quy mô vừa và nhỏ, mô hình quản lý thủ công truyền thống sử dụng sổ sách hoặc các phần mềm desktop cục bộ (Stand-alone) lỗi thời đã bộc lộ nhiều hạn chế chí mạng. Các hệ thống cũ gặp khó khăn lớn trong việc đồng bộ hóa dữ liệu giữa các quầy giao dịch, không có khả năng mở rộng kiến trúc khi số lượng đầu sách tăng lên, và đặc biệt là trải nghiệm người dùng của thủ thư cực kỳ nghèo nàn do giao diện phản hồi chậm. Việc thiếu hụt một cơ chế tự động ràng buộc nghiệp vụ dẫn đến tình trạng sai sót trong việc tính toán thời hạn mượn trả, thất thoát tài nguyên sách và gây ra sự chậm trễ trong quy trình quyết toán tài chính khi độc giả phát sinh nợ phạt trễ hạn.

Để giải quyết triệt để các bài toán thực tiễn nêu trên, đề án môn học Lập trình hướng đối tượng của nhóm tập trung nghiên cứu, thiết kế và triển khai Hệ thống Quản lý Thư viện (Lib-Management). Điểm sáng đổi mới công nghệ của hệ thống nằm ở việc áp dụng mô hình kiến trúc tách biệt hoàn toàn giữa Tầng giao diện người dùng và Tầng lõi xử lý nghiệp vụ (Decoupled Architecture), một xu hướng tối ưu hóa hệ thống Web đã được minh chứng bởi Cerny và các cộng sự [3]. Sự kết hợp này mang lại khả năng phân tách vai trò rõ rệt: sử dụng nền tảng Java Spring Boot để xây dựng một lõi dịch vụ API bền vững theo chuẩn RESTful của Fielding [4]; đồng thời kết hợp với thư viện React.js để dựng nên giao diện Single Page Application (SPA) tương tác bất đồng bộ thời gian thực mượt mà. Bên cạnh đó, cốt lõi của mã nguồn Backend được cấu trúc chặt chẽ dựa trên các nguyên lý lập trình hướng đối tượng (OOP) nâng cao và tư duy mã nguồn sạch của Martin [5], bao gồm Tính Đa hình động (Polymorphism) và Mẫu thiết kế chiến lược (Strategy Pattern) định hình bởi Gamma [6] cho phân hệ tài chính. Hệ thống thực hiện lưu trữ dữ liệu thông qua cơ chế ánh xạ tệp cứng cấu trúc JSON, giúp đơn giản hóa hạ tầng triển khai nhưng vẫn đảm bảo tính toàn vẹn dữ liệu.

Báo cáo này được cấu trúc một cách logic nhằm trình bày toàn bộ vòng đời phát triển của dự án phần mềm. Nội dung báo cáo sẽ đi sâu phân tích từ bước khảo sát bài toán, phân tích yêu cầu nghiệp

vụ chuyên sâu, thiết kế kiến trúc hệ thống và sơ đồ lớp chi tiết, cho đến kỹ thuật triển khai mã nguồn cốt lõi và các kịch bản kiểm thử nghiệm thu thực tế hệ thống.

CHƯƠNG 1

TỔNG QUAN VỀ BÀI TOÁN VÀ ĐỀ TÀI ĐẶT RA

1.1 Lý do chọn đề tài

1.1.1 Thực trạng quản lý lưu thông sách tại các thư viện quy mô vừa và nhỏ

Tại các thư viện trường học thuộc phân khoa hoặc các trung tâm tư liệu nhỏ, quy trình quản lý lưu thông sách hiện nay vẫn mang nặng tính thủ công hoặc sử dụng các công cụ tin học văn phòng rời rạc. Việc ghi nhận thông tin mượn trả trên sổ sách hoặc các cơ sở dữ liệu phẳng cục bộ thường xuyên dẫn đến tình trạng bất đồng bộ dữ liệu. Theo phân tích về các hệ thống thông tin lỗi thời của Sommerville [1], khi thủ thư thực hiện các tác vụ kiểm kho, việc đối chiếu giữa số lượng sách thực tế trên giá và số lượng sách đang cho mượn mất rất nhiều thời gian và dễ xảy ra sai sót. Thêm vào đó, việc thiếu hụt một hệ thống cảnh báo tự động về thời hạn trả sách khiến cho tỷ lệ độc giả giữ sách quá hạn tăng cao, gây ảnh hưởng trực tiếp đến quyền lợi tiếp cận tài liệu của các độc giả khác và làm giảm hiệu suất quay vòng của tài nguyên thư viện.

1.1.2 Nhu cầu tối ưu hóa tốc độ xử lý dữ liệu và trải nghiệm quản trị thời gian thực

Trước sự phát triển của công nghệ Web hiện đại, người dùng phần mềm – cụ thể là các thủ thư – đòi hỏi hệ thống phải có khả năng phản hồi ngay lập tức đối với mọi thao tác nghiệp vụ. Khi một độc giả thực hiện trả sách, số lượng tồn kho của đầu sách đó phải được cộng trả ngay lập tức, và trạng thái phiếu mượn phải chuyển sang chế độ quyết toán theo thời gian thực (Real-time). Để đạt được tốc độ này, việc dịch chuyển từ kiến trúc Monolithic cứng kênh sang kiến trúc tách biệt (Decoupled Architecture) là một xu thế tất yếu trong kỹ nghệ phần mềm hiện đại [3]. Việc phân tách Frontend và Backend giúp giải phóng áp lực tính toán cho trình duyệt, cho phép Backend tập trung xử lý logic nghiệp vụ và trả dữ liệu thô dưới dạng JSON, giúp tối ưu hóa băng thông đường truyền và đẩy cao trải nghiệm quản trị lên mức tối đa.

1.1.3 Thách thức áp dụng nguyên lý OOP vào giải quyết bài toán thực tế

Một trong những thách thức lớn nhất của đề tài là việc chuyển dịch các quy tắc, ràng buộc của thế giới thực vào trong cấu trúc mã nguồn một cách tường minh và khoa học dựa trên tư duy hướng đối tượng. Bài toán quản lý thư viện đòi hỏi phải phân loại nhiều nhóm độc giả khác nhau như Sinh viên thường, Sinh viên ưu tiên và Giảng viên, trong đó mỗi nhóm lại có các quyền lợi về hạn mức số lượng sách và thời gian mượn hoàn toàn khác biệt.

Hơn thế nữa, bài toán tính toán tài chính khi trễ hạn trả sách đòi hỏi hệ thống phải vận hành cực kỳ linh hoạt; nếu áp dụng các câu lệnh rẽ nhánh if-else truyền thống sẽ dẫn đến hiện tượng mã nguồn bị "bốc mùi" (Code Smell) và rất khó bảo trì theo nguyên lý đóng mở (Open-Closed Principle) của Martin [7]. Do đó, việc nghiên cứu áp dụng Tính Đa hình để cấu trúc lớp con và Mẫu thiết kế Chiến lược (Strategy Pattern) để cô lập thuật toán tính phí phạt định nghĩa bởi Freeman [8] là một thách thức kỹ thuật lớn, giúp chứng minh giá trị thực tiễn của các lý thuyết OOP chuyên sâu.

1.2 Mục tiêu của dự án

Dự án được triển khai nhằm đạt được ba mục tiêu cốt lõi sau đây, đại diện cho ba tầng kiến trúc của một hệ thống phần mềm tiêu chuẩn:

1. Xây dựng ứng dụng hoàn chỉnh phân tầng rõ rệt: Mục tiêu kỹ thuật đầu tiên là thiết lập một kiến trúc Decoupled chuẩn mực theo mô hình Clean Architecture [7]. Tầng giao diện (Frontend React SPA) sử dụng mô hình lập trình hàm và Hook để quản lý trạng thái, thực hiện các tương tác mượt mà và gọi API bất đồng bộ thông qua Fetch API tuân thủ tài liệu đặc tả của Meta [9]. Tầng nghiệp vụ (Backend Spring Boot) đóng vai trò là một Stateless REST API, tiếp nhận requests, xử lý dữ liệu tập trung và điều phối luồng thông tin an toàn theo tài liệu hướng dẫn của Spring Team [10].
2. Triển khai thành công các tính năng CRUD lõi hệ thống: Hiện thực hóa trọn vẹn và tối ưu các module chức năng bao gồm:
 - Quản lý độc giả đa hình với khả năng tự động nhận diện nhóm đối tượng khi nạp dữ liệu dựa trên đặc tả cấu hình của FasterXML [11].
 - Quản lý kho sách với các ràng buộc về mã định danh và tính hợp lệ của số lượng tồn kho thông qua BookRequestDTO.
 - Hệ thống Sổ giám sát lưu thông sách (BorrowRecord) phẳng hóa dữ liệu giúp thủ thư theo dõi trực quan trạng thái mượn trả và thực hiện tác vụ "Trả nhanh" chỉ với một cú click chuột.
3. Đảm bảo tính toàn vẹn dữ liệu không phụ thuộc RDBMS công kênh: Dự án đặt mục tiêu tự xây dựng một Tầng lưu trữ dữ liệu (Persistence Layer) gọn nhẹ bằng cách đọc/ghi trực tiếp xuống các file cứng cấu trúc JSON thông qua thư viện Jackson ObjectMapper. Hệ thống áp dụng nguyên lý đảo ngược phụ thuộc (Dependency Injection) của Fowler [12] để tự kiểm soát tính toàn vẹn liên kết trên bộ nhớ RAM bằng Java Collections Framework [13], đảm bảo rằng không thể xóa một độc giả

nếu họ vẫn đang nợ sách hoặc chưa hoàn thành phí phạt, duy trì cấu trúc dữ liệu luôn ở trạng thái nhất quán.

1.3 Phạm vi áp dụng và đối tượng nghiên cứu

1.3.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề án bao gồm hai thành phần thực thể cốt lõi trong không gian bài toán quản lý thư viện được phân tích theo mô hình thực thể hướng đối tượng của Sommerville [1]:

- Thực thể Kho Sách: Nghiên cứu các thuộc tính định danh cơ bản của sách (Mã sách, tiêu đề, tác giả, thể loại, đơn giá) và trạng thái động của sách (số lượng tồn kho hiện hành trong tệp JSON) dưới tác động của luồng nghiệp vụ lưu thông.
- Thực thể Độc Giả: Nghiên cứu mô hình phân cấp hành vi của ba nhóm đối tượng đặc thù bao gồm Sinh viên thường, Sinh viên ưu tiên và Giảng viên. Tập trung làm rõ sự khác biệt về mặt thuộc tính (hạn mức mượn tối đa) và hành vi (thuật toán tính toán biên độ phí phạt trễ hạn trên từng ngày quá hạn) bằng tính đa hình [13].

1.3.2 Phạm vi áp dụng

Hệ thống được thiết kế và tối ưu hóa để triển khai thực nghiệm trong phạm vi mạng nội bộ (Intranet) của các tổ chức, trường học hoặc phân khoa có quy mô vừa và nhỏ theo các tiêu chuẩn bảo mật hệ thống cục bộ của Pressman [2]. Phần mềm hoạt động theo mô hình Client-Server độc lập, giao tiếp xuyên qua các chính sách bảo mật CORS cấu trúc bởi MDN [14], đóng vai trò là công cụ quản trị chuyên trách dành riêng cho thủ thư vận hành tác vụ trực tiếp tại quầy lưu thông sách.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1 Bốn nguyên lý cốt lõi của Lập trình hướng đối tượng (OOP)

Kiến trúc phần mềm của hệ thống Quản lý Thư viện được xây dựng dựa trên sự vận hành phối hợp và chặt chẽ của bốn nguyên lý nền tảng trong lập trình hướng đối tượng (OOP), tuân thủ các tiêu chuẩn thiết kế hệ thống bền vững [13, 5].

2.1.1 Tính đóng gói (Encapsulation)

Tính đóng gói được thể hiện qua việc cô lập trạng thái nội tại và thuộc tính nhạy cảm của thực thể bên trong lớp, ngăn chặn các tác động thay đổi dữ liệu bất hợp lý từ các tác nhân bên ngoài [13]. Trong hệ thống, tất cả các trường dữ liệu cốt lõi như mã sách (bookId), tiêu đề (title), hạn mức (quantity), hay thông tin giao dịch (BorrowRecord) đều được khai báo với phạm vi truy cập nghiêm ngặt private.

Việc truy xuất và điều chỉnh dữ liệu bắt buộc phải đi qua các cổng giao tiếp công khai là hệ thống các hàm Getter và Setter chuẩn hóa. Cách thiết kế này cho phép hệ thống tích hợp bộ lọc kiểm tra dữ liệu đầu vào (Data Validation) trước khi ghi nhận vào thuộc tính đối tượng, bảo đảm tính an toàn dữ liệu và che giấu cấu trúc cài đặt hạ tầng kỹ thuật bên dưới [5].

2.1.2 Tính kế thừa (Inheritance)

Nguyên lý kế thừa giúp tối ưu hóa cấu trúc mã nguồn, tăng cường khả năng tái sử dụng mã và thiết lập mối quan hệ phân cấp rõ ràng giữa các thực thể trong không gian bài toán [1]. Hệ thống định nghĩa một lớp cơ sở trừu tượng đóng vai trò nền móng là User, tập trung các thuộc tính cơ bản của một người dùng hệ thống (Mã người dùng, họ tên, số điện thoại).

Lớp Reader kế thừa từ lớp User để mở rộng vai trò bạn đọc, và từ lớp Reader, hệ thống tiếp tục phân nhánh kế thừa ra 3 lớp con cụ thể: StudentReader, PriorityStudentReader, và LecturerReader. Cấu trúc cây kế thừa này giúp các phân lớp chuyên biệt tự động sở hữu toàn bộ các thuộc tính nền tảng của cha mà không cần khai báo lại, đồng thời dễ dàng mở rộng thêm các logic nghiệp vụ đặc thù cho từng loại thể đọc giả.

2.1.3 Tính đa hình (Polymorphism)

Tính đa hình là cơ chế cốt lõi mang lại sự linh hoạt tối đa cho hệ thống khi cho phép xử lý các đối tượng thuộc các phân lớp con khác nhau thông qua một giao diện định danh chung tại thời điểm thực thi (Runtime) [13]. Trong đồ án này, tính đa hình được hiện thực hóa sâu sắc qua hai khía cạnh:

- Cơ chế nạp dữ liệu đa hình Jackson Polymorphic Deserialization: Hệ thống sử dụng các annotation nâng cao của thư viện FasterXML bao gồm `@JsonTypeInfo` và `@JsonSubTypes` đặt trên đầu lớp cha Reader [11]. Khi đọc tệp cứng JSON, Jackson tự động bóc tách chuỗi định danh Tiếng Việt có dấu (như "Sinh viên ưu tiên", "Giảng viên") để ép kiểu và khởi tạo chính xác thực thể lớp con tương ứng trên bộ nhớ RAM một cách tự động.
- Đa hình trong tính toán quyết toán tiền phạt trễ hạn: Lệnh gọi hàm tính toán phí phạt phát ra từ tầng Service được áp dụng chung cho mọi đối tượng kiểu Reader, nhưng hành vi xử lý tài chính thực tế dưới hạ tầng sẽ tự động thay đổi dựa trên cấu hình lớp con thực tế của bạn đọc đó tại runtime.

2.1.4 Tính trừu tượng (Abstraction)

Tính trừu tượng giúp hệ thống cô lập các chi tiết cài đặt thuật toán phức tạp ở các tầng sâu bên dưới, chỉ phơi bày ra các khung hành vi tổng quát ở tầng bề mặt cho các module khác gọi sử dụng [7]. Hệ thống thiết lập tính trừu tượng thông qua việc xây dựng lớp Abstract Class Reader (đóng vai trò định hình bộ khung thông tin bạn đọc mà không cho phép khởi tạo trực tiếp lớp này) và Interface FinePolicy.

Thông qua việc khai báo chữ ký hàm trong interface FinePolicy, hệ thống quy định một cam kết bắt buộc về hành vi cho toàn bộ các lớp con bên dưới. Tầng dịch vụ nghiệp vụ trung tâm (BorrowReturnService) khi gọi tính tiền phạt chỉ cần tương tác trực tiếp với tầng trừu tượng này mà hoàn toàn không cần quan tâm hay phụ thuộc vào chi tiết cài đặt thuật toán tính toán cụ thể bên trong của từng loại thẻ, giúp giảm thiểu tối đa độ phụ thuộc chéo (Loose Coupling) giữa các thành phần kiến trúc [5].

2.2 Mẫu thiết kế Chiến lược (Strategy Pattern) trong tính toán tiền phạt

Trong quá trình vận hành nghiệp vụ lưu thông thư viện, bài toán xử lý phạt tài chính khi đọc giả hoàn trả sách trễ hạn thường xuyên biến động và phụ thuộc chặt chẽ vào chính sách ưu đãi của từng loại đối tượng. Nếu áp dụng các cấu trúc rẽ nhánh truyền thống (if-else hoặc switch-case) lồng nhau tại tầng nghiệp vụ, hệ thống sẽ vi phạm nguyên lý Đóng-Mở (Open-Closed Principle) trong thiết kế hướng đối tượng, khiến mã nguồn trở nên cồng kềnh và cực kỳ khó bảo trì khi phát sinh thêm loại thẻ mới [7].

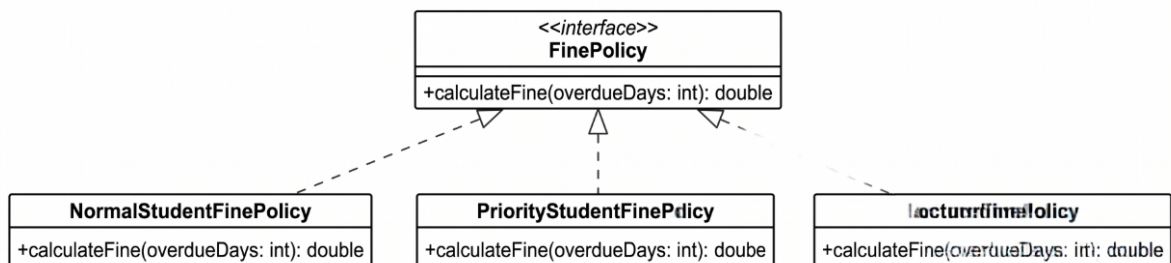
Để giải quyết triệt để vấn đề này, hệ thống triển khai mẫu thiết kế Chiến lược (Strategy Pattern) nhằm tách biệt hoàn toàn logic tính toán tiền phạt trễ hạn ra khỏi lớp nghiệp vụ chính [6, 8]. Cấu trúc thiết kế mẫu bao gồm:

- Giao diện Chiến lược (Strategy Interface): Định nghĩa một interface trừu tượng FinePolicy chứa

chữ ký hàm duy nhất là `double calculateFine(int overdueDays)` [8].

- Các Chiến lược Cụ thể (Concrete Strategies): Bao gồm ba lớp `NormalStudentFinePolicy` (phạt theo hệ số cơ bản trên mỗi ngày trễ hạn), `PriorityStudentFinePolicy` (áp dụng hệ số giảm trừ ưu tiên cho sinh viên có hoàn cảnh đặc biệt), và `LecturerFinePolicy` (thiết lập thời gian ân hạn dài hơn và mức tính phí đặc thù dành riêng cho cán bộ giảng viên).

Mỗi thực thể thuộc lớp `Reader` khi được nạp lên RAM sẽ sở hữu một tham chiếu đến một chiến lược cụ thể tương ứng thông qua cơ chế tổng hợp (Aggregation) [6]. Khi thực hiện hoàn trả sách qua form hoặc click nút "Trả nhanh" trên dòng bảng giao diện, hệ thống tự động đối chiếu `LocalDate` hôm nay với hạn trả (`dueDate`). Nếu phát sinh trễ hạn, tầng `Service` chỉ cần phát lệnh thực thi hàm tính toán mà không cần quan tâm thực thể đang xử lý là sinh viên hay giảng viên, giúp mã nguồn đạt độ linh hoạt tối đa và dễ dàng mở rộng trong tương lai [8].



Hình 2.1: Sơ đồ cấu trúc Strategy Pattern FinePolicy

2.3 Kiến trúc tách biệt Frontend - Backend (Decoupled Architecture)

Hệ thống Quản lý Thư viện được xây dựng theo mô hình kiến trúc tách biệt hình thái (Decoupled Architecture), tách rời hoàn toàn vai trò vận hành và triển khai giữa Máy chủ cung cấp dịch vụ dữ liệu và Máy trạm xử lý giao diện người dùng [3].

2.3.1 Backend (Java Spring Boot REST API)

Tầng Backend được xây dựng trên nền tảng Spring Boot framework v3.2.5 vững chắc, vận hành máy chủ nhúng Apache Tomcat tại cổng 8080 [10]. Backend đóng vai trò là một máy cung cấp dữ liệu thuần túy (Stateless API), chịu trách nhiệm thực thi các thuật toán kiểm tra ràng buộc toàn vẹn dữ liệu, bẫy lỗi tập trung qua `GlobalExceptionHandler`, và xử lý nghiệp vụ lưu thông sách. Toàn bộ các yêu cầu tương tác từ bên ngoài gửi đến đều được tiếp nhận và phản hồi hoàn toàn dưới dạng cấu trúc chuỗi thô JSON thông qua các phương thức HTTP Methods tiêu chuẩn [4]:

- GET: Truy xuất danh mục toàn bộ kho sách, đọc giả và lịch sử lưu thông.
- POST: Đăng ký cấp thẻ bạn đọc hoặc khởi tạo phiếu mượn sách mới.
- PUT: Hiệu chỉnh cập nhật thông tin chi tiết của đầu sách hoặc tài khoản.

- DELETE: Dọn dẹp kho dữ liệu hoặc hủy thẻ bạn đọc (bắt buộc qua bộ lọc bảo vệ an toàn).

2.3.2 Frontend (React.js SPA)

Tầng Frontend đảm nhận vai trò quản lý trạng thái giao diện người dùng (State management), được xây dựng dưới dạng ứng dụng đơn trang Single Page Application (SPA) thông qua thư viện React v18+ và công cụ đóng gói siêu tốc Vite Bundler chạy tại cổng 5173 [9]. Frontend chịu trách nhiệm xây dựng giao diện Dashboard Responsive, thu thập thông tin biểu mẫu nhập liệu và kết xuất dữ liệu trực quan lên các hàng bảng.

Mọi tác vụ giao tiếp dữ liệu với máy chủ Backend đều được thực hiện hoàn toàn bất đồng bộ (Asynchronous) thông qua cơ chế Fetch API của JavaScript [15]. Nhờ cơ chế này, khi thủ thư thực hiện các thao tác thêm sách hay click nút "Trả nhanh", giao diện chỉ tải lại duy nhất thành phần bảng dữ liệu liên quan mà không cần phải nạp lại toàn bộ trang Web, tối ưu hóa tốc độ phản hồi và trải nghiệm người dùng lên mức tối đa [9].

2.3.3 Cơ chế CORS (Cross-Origin Resource Sharing)

Do Frontend (<http://localhost:5173>) và Backend (<http://localhost:8080>) vận hành tại hai nguồn gốc (Origins) khác nhau về mặt cổng mạng (Ports), theo cơ chế bảo mật nghiêm ngặt của trình duyệt web (Same-Origin Policy), mọi yêu cầu gọi API bất đồng bộ từ React sang Spring Boot mặc định sẽ bị bảo mật chặn lại [14].

Để giải quyết rào cản này, hệ thống cấu hình bộ lọc chống chặn CORS tập trung tại tầng Backend. Máy chủ Spring Boot được thiết lập để đính kèm các thẻ tiêu đề HTTP Headers an toàn cho phép nguồn từ cổng 5173 được quyền truy xuất và thực thi đầy đủ các phương thức cấu trúc dữ liệu, tạo điều kiện cho luồng thông tin xuyên suốt giữa hai nền tảng kiến trúc [10, 14].

2.4 Công nghệ lưu trữ dữ liệu phi quan hệ bằng tệp tin cứng JSON

Đối với các dự án phần mềm có quy mô vừa và nhỏ, việc cấu hình và duy trì một hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) như MySQL hay PostgreSQL thường mang lại sự cồng kềnh không cần thiết trong khâu triển khai và di động mã nguồn [1]. Hệ thống Quản lý Thư viện lựa chọn giải pháp lưu trữ phi quan hệ trực tiếp xuống hệ thống các tệp tin cứng định dạng JSON (readers.json, books.json, borrow_tickets.json) đặt ngay tại thư mục data/ của dự án. Phương pháp này mang lại ưu điểm vượt trội về tính gọn nhẹ, dễ dàng sao lưu, di chuyển toàn bộ hệ thống sang máy tính khác mà không cần cài đặt lại hạ tầng cơ sở dữ liệu nền bên dưới.

Để thực hiện cầu nối đồng bộ dữ liệu giữa các tệp tin vật lý này với các đối tượng thực thể lập trình, hệ thống áp dụng bộ thư viện chuyển đổi Jackson ObjectMapper chuẩn công nghiệp [11]. Vai trò của tầng Repository được cấu trúc như một Persistence Layer tự chế vận hành theo hai luồng nghiệp vụ cốt lõi:

1. Luồng Deserialize (Load từ Disk lên RAM): Khi ứng dụng Spring Boot được kích hoạt khởi chạy, ObjectMapper sẽ tiến hành đọc toàn bộ cấu trúc chuỗi văn bản từ các file JSON trong thư mục data/, tự động biên dịch, ép kiểu đa hình và nạp dữ liệu vào các cấu trúc mảng Java Collections (như List, Map) nằm trên bộ nhớ RAM để phục vụ cho tầng Service truy xuất với tốc độ tức thời [13, 11].
2. Luồng Serialize (Save từ RAM xuống Disk): Bất cứ khi nào hệ thống phát sinh các tác vụ làm thay đổi trạng thái dữ liệu (Thêm mới độc giả, hiệu chỉnh số lượng tồn kho sách khi mượn, hay đóng phiếu trả sách), tầng Repository sẽ lập tức kích hoạt hàm saveToFile() ngầm. ObjectMapper thực hiện tuần tự hóa toàn bộ danh sách đối tượng Java từ bộ nhớ RAM thành chuỗi cấu trúc JSON và ghi đè an toàn xuống tệp tin cứng vật lý trên ổ đĩa [11]. Cơ chế này bảo đảm dữ liệu của thư viện luôn được lưu trữ vĩnh viễn và không bị thất thoát kể cả khi hệ thống máy chủ API tắt hoặc khởi động lại đột ngột.

CHƯƠNG 3

PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Phân tích yêu cầu chức năng (Use Case Analysis)

Dựa trên các quy trình nghiệp vụ thực tế tại quầy lưu thông thư viện, hệ thống được cấu trúc hóa để đáp ứng trọn vẹn các yêu cầu chức năng cốt lõi thông qua mô hình ca sử dụng (Use Case), chia tách thành ba phân hệ phân tầng độc lập phục vụ trực tiếp cho tác vụ quản trị của thủ thư [1]:

3.1.1 Phân hệ Quản lý thực thể Danh mục (Core CRUD)

- **Quản lý Kho Sách:** Thủ thư có quyền thực hiện thêm mới đầu sách vào kho hệ thống, hiệu chỉnh các thuộc tính chi tiết (tiêu đề, tác giả, đơn giá) và loại bỏ đầu sách khi tài nguyên bị thanh lý hoặc hư hỏng. Hệ thống tích hợp bộ lọc tự động kiểm tra trùng lặp mã định danh duy nhất và xác thực định mức tồn kho trước khi đồng bộ xuống ổ cứng.
- **Quản lý Tài khoản Độc giả:** Hỗ trợ đăng ký cấp mới thẻ bạn đọc, phân cấp cấu hình thẻ sang các nhóm đối tượng chuyên biệt và cập nhật thông tin liên lạc (Họ tên, Số điện thoại). Hệ thống quản trị chặt chẽ quyền hạn dựa trên mô hình hình thái học để thiết lập các bẫy logic giới hạn ở các bước tiếp theo.

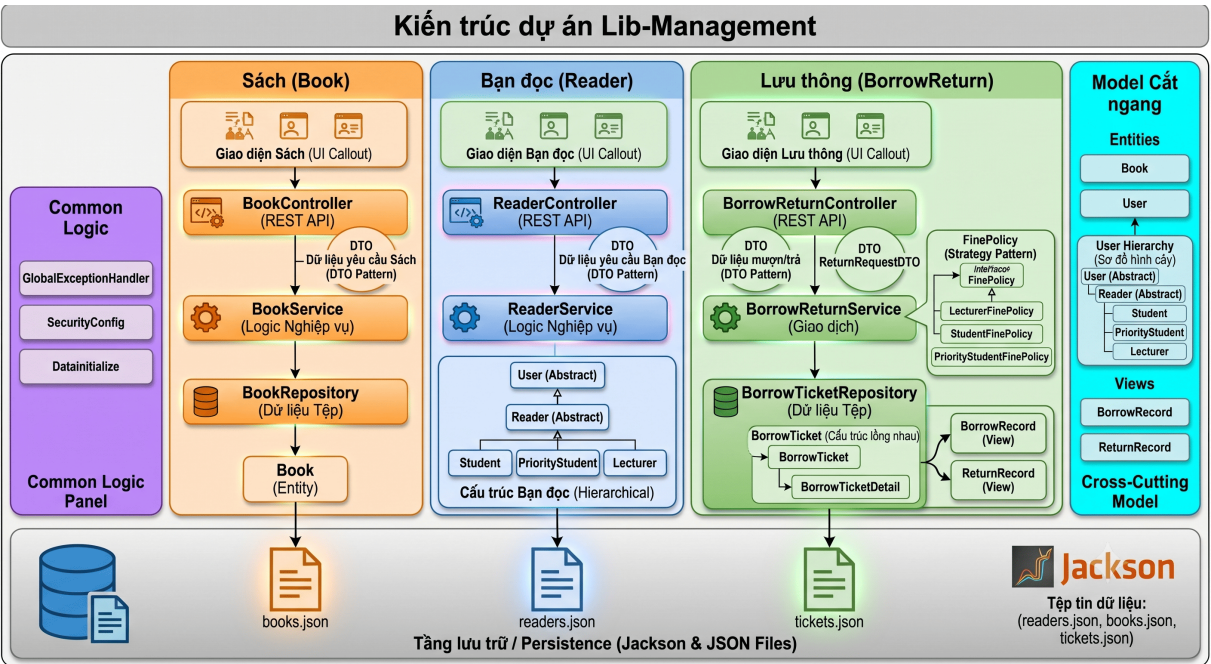
3.1.2 Phân hệ Điều phối Lưu thông Sách (Transaction Management)

- **Đăng ký Giao dịch Mượn Sách:** Tiếp nhận mã độc giả và danh sách mã sách yêu cầu. Hệ thống thực hiện chuỗi kiểm tra điều kiện nghiêm ngặt: xác thực sự tồn tại của tài khoản, kiểm tra định mức số lượng sách còn lại trong kho vật lý, và đối chiếu tổng số sách độc giả đang cầm giữ để đảm bảo không vượt quá giới hạn tối đa cho phép của loại thẻ tương ứng.
- **Quyết toán Giao dịch Trả Sách:** Thu hồi tài nguyên sách, đóng trạng thái phiên giao dịch hiện hành và cập nhật hoàn trả số lượng tồn kho vật lý. Phân hệ chịu trách nhiệm tính toán khoảng cách thời gian và kích hoạt cơ chế tài chính phạt nếu phát sinh quá hạn.

3.1.3 Phân hệ Giám sát Luồng Dữ liệu (Real-time Monitoring)

Cung cấp màn hình Dashboard trung tâm phẳng hóa, kết xuất trực quan toàn bộ các phiên mượn sách chưa hoàn trả đang hoạt động trong hệ thống. Module này cho phép thủ thư thực hiện thao tác duyệt nhanh, truy xuất thông tin hạn trả để đưa ra nhắc nhở và kích hoạt nút quyết toán nhanh trực tiếp trên từng dòng dữ liệu của bảng hiển thị.

3.2 Thiết kế Sơ đồ lớp chi tiết (Detailed Class Diagram Analysis)



Hình 3.1: Giao diện phân hệ Quản lý thông tin và số lượng tồn kho Sách

Cấu trúc mã nguồn của hệ thống được hiện thực hóa dựa trên sơ đồ lớp chi tiết trong tài liệu đặc tả HeThongQuanLyThuVien.puml nhằm đảm bảo tính bao đóng và kế thừa trong thiết kế cấu trúc phần mềm hướng đối tượng [13, 5]. Dự án triển khai mô hình phân tách thành 6 cấu trúc phân tầng chức năng độc lập nhằm đạt được tiêu chuẩn mã nguồn sạch (Clean Code) và giảm thiểu tối đa sự phụ thuộc chéo [5, 7]:

3.2.1 Tầng Thực thể Nghiệp vụ (Domain Models Layer)

Đây là tầng trung tâm định nghĩa cấu trúc dữ liệu cốt lõi của không gian bài toán, áp dụng triệt để nguyên lý đóng gói bằng cách thiết lập phạm vi truy cập private cho tất cả các trường thuộc tính và cung cấp hệ thống hàm Getter/Setter để kiểm soát dữ liệu đầu ra [13]:

- Cấu trúc Phân cấp Độc giả: Khởi tạo từ lớp cha trừu tượng User chứa các thông tin cơ bản (userId, fullName, phoneNumber) [13], hệ thống mở rộng ra lớp abstract Reader chứa tham chiếu đa hình đến chiến lược tài chính FinePolicy dựa trên nguyên lý thiết kế hướng đối tượng bền vững

[6, 8]. Từ Reader, hệ thống triển khai 3 lớp con cụ thể: StudentReader, PriorityStudentReader, và LecturerReader phục vụ cho cơ chế tuần hoàn tự hóa dữ liệu đa hình [11, 13]. Các lớp con này chịu trách nhiệm định nghĩa các giá trị hằng số đóng gói về giới hạn mượn tối đa (MAX_BORROW_LIMIT) và định danh chuỗi phân loại (READER_TYPE) để tường minh hóa mã nguồn [5, 13].

- Thực thể Kho Sách (Book): Lưu trữ thông tin chi tiết của một đầu sách bao gồm mã định danh bookId, title, author, category, số lượng tồn kho quantity, và đơn giá vật lý price [13].
- Thực thể Phiếu Giao dịch (BorrowTicket & BorrowTicketDetail): Lớp BorrowTicket quản lý thông tin tổng quan của một phiên lưu thông, bao gồm ticketId, mã độc giả readerId, các mốc thời gian (borrowDate, dueDate, returnDate), trạng thái giao dịch thông qua kiểu liệt kê TicketStatus (BORROWING, RETURNED) và số tiền phạt đã nộp finePaid [13]. Để tối ưu hóa cấu trúc dữ liệu theo nguyên lý chuẩn hóa, chi tiết danh mục sách mượn được tách nhỏ thành một danh sách lồng nhau kiểu BorrowTicketDetail, quản lý cặp thuộc tính gồm mã sách bookId và số lượng mượn quantity tối ưu trên bộ nhớ [13].
- Thực thể Kết xuất Giao diện (BorrowRecord & ReturnRecord): Lớp BorrowRecord đóng vai trò phẳng hóa cấu trúc dữ liệu đa tầng từ phiếu mượn, tích hợp các thuộc tính readerId, bookId, quantity, borrowDate, và dueDate để phục vụ truyền tải luồng thông tin đồng bộ dữ liệu sang biểu đồ máy trạm Single Page Application [9]. Lớp ReturnRecord lưu giữ vết của các phiên hoàn trả thành công phục vụ công tác thống kê và báo cáo [5].

3.2.2 Tầng Truy xuất và Lưu trữ Dữ liệu (Persistence / Repositories Layer)

Tầng này chịu trách nhiệm quản lý trực tiếp luồng đọc/ghi dữ liệu xuống ổ cứng vật lý thông qua thư viện Jackson ObjectMapper, cô lập toàn bộ thao tác tệp tin khỏi các tầng dịch vụ phía trên [11]:

- BookRepository: Quản lý danh sách đối tượng Book nạp từ tệp cứng books.json, hỗ trợ các hàm getAll(), findById(), addBook(), deleteBook(), và cập nhật trạng thái kho update() đồng bộ dữ liệu [11].
- ReaderRepository: Quản lý danh sách tài khoản độc giả từ tệp readers.json, triển khai các tác vụ truy xuất và lưu trữ tương ứng [11].
- BorrowTicketRepository: Quản lý lịch sử giao dịch lưu thông từ tệp borrow_tickets.json, tích hợp bộ sinh mã tự động generateNextId() bằng biến an toàn luồng AtomicInteger nhằm ngăn chặn hiện tượng tranh chấp vùng nhớ và xung đột chỉ mục khi có nhiều yêu cầu đồng thời [13].

3.2.3 Tầng Logic Nghiệp vụ Trung tâm (Business Services Layer)

Nơi tập trung toàn bộ các thuật toán điều phối luồng thông tin, xử lý tính toán và áp dụng các ràng buộc nghiệp vụ phân tầng độc lập của hệ thống [7]:

- BookService & ReaderService: Thực hiện các tác vụ kiểm tra tính hợp lệ của dữ liệu đầu vào, xác thực định dạng thông tin trước khi chuyển tiếp dữ liệu sang tầng Repository [5].

- `BorrowReturnService`: Lớp điều khiển lưu thông của toàn bộ dự án kiến trúc lõi [7]. Lớp này chứa các phương thức nghiệp vụ phức tạp bao gồm `borrowBook()` (xử lý logic mượn sách, tự động trừ kho và kiểm tra hạn mức thẻ) [5], `returnBook()` (thu hồi tài nguyên, cập nhật trạng thái phiếu, tính toán khoảng cách ngày quá hạn và trả về số tiền phạt kiểu long cho API) [13], và `getAllRecords()` (quét danh sách phiếu mượn, bóc tách cấu trúc mảng lồng nhau để trả về danh sách phẳng `BorrowRecord` truyền sang Frontend) [10].

3.2.4 Tầng Đối tượng Chuyển đổi Dữ liệu (Data Transfer Objects - DTO)

Nhằm mục đích bảo mật kiến trúc hệ thống và ngăn chặn hiện tượng phơi bày các thực thể mô hình gốc ra môi trường Internet, dự án thiết lập một phân tầng DTO đóng vai trò như những biên giới an toàn bảo vệ luồng dữ liệu truyền tải [5, 7]:

- `BookRequestDTO` & `ReaderRequestDTO`: Đóng gói thông tin yêu cầu khởi tạo hoặc cập nhật từ Frontend gửi về, cô lập hoàn toàn các trường dữ liệu nhạy cảm [5, 7].
- `BorrowRequestDTO` & `ReturnRequestDTO`: Đóng gói dữ liệu tối giản phục vụ riêng cho các cổng giao dịch mượn/trả sách (chỉ chứa mã độc giả, mã sách và số lượng), giảm thiểu tối đa băng thông mạng [5].

3.2.5 Tầng Chiến lược Tài chính Đa hình (Polymorphic Strategy Pattern)

Hiện thực hóa trọn vẹn mẫu thiết kế Chiến lược (Strategy Pattern) để xử lý bài toán biến động chính sách phạt tài chính khi độc giả hoàn trả sách quá hạn quy định [6, 8]. Hệ thống định nghĩa một giao diện chung là interface `FinePolicy` chứa chữ ký hàm `double calculateFine(int overdueDays)` để phân tách cấu trúc nghiệp vụ [6, 8].

Ba lớp chiến lược cụ thể bao gồm `NormalStudentFinePolicy`, `PriorityStudentFinePolicy`, và `LecturerFinePolicy` tiến hành hiện thực hóa interface này để cài đặt các giải thuật tính phí phạt biệt lập đặc thù của riêng mình [6, 8]. Cơ chế kế thừa kết hợp đa hình giúp tầng Service gọi thực thi tính phạt một cách thống nhất thông qua lớp cha trừu tượng mà không bị ràng buộc vào chi tiết thuật toán bên dưới [13].

3.2.6 Tầng Định tuyến API và Bẫy lỗi Tập trung (Controllers & Web Security Boundary)

- `LibraryApiController`: Tầng biên giới tiếp nhận toàn bộ các yêu cầu HTTP Requests bất đồng bộ từ giao diện React gửi đến thông qua cấu hình Spring Web Beans [10]. Lớp này định nghĩa các Endpoint RESTful chuẩn hóa (như `/api/v1/books`, `/api/v1/borrow`), thực hiện giải nén các đối tượng DTO, chuyển giao cho tầng Service xử lý và đóng gói kết quả trả về dưới dạng `ResponseEntity` chứa mã trạng thái HTTP Status phù hợp [4, 10].
- `GlobalExceptionHandler`: Phân hệ quản lý bẫy lỗi tập trung ứng dụng cơ chế Aspect-Oriented

Programming (AOP) của Spring Framework [10]. Lớp này chịu trách nhiệm đánh chặn và xử lý toàn bộ các Custom Checked Exceptions phát sinh từ tầng nghiệp vụ sâu bên dưới như `DataNotFoundException`, `OutOfStockException`, và `BorrowLimitExceededException` [13, 10]. Hệ thống tự động phân tích lỗi và cấu trúc lại thông điệp cảnh báo thành chuỗi JSON mạch lạc gửi về giao diện người dùng, ngăn chặn tình trạng sập server hoặc hiển thị mã lỗi thô mắt thẩm mỹ.

- `SecurityConfig`: Thiết lập cấu trúc bộ lọc an ninh mạng, cho phép định tuyến luồng thông tin thông suốt và mở cổng giao tiếp CORS an toàn cho máy trạm React [10, 14].

3.3 Thiết kế Luồng xử lý Dữ liệu và Cấu trúc Lưu trữ Tập JSON

Mô hình lưu trữ dữ liệu của hệ thống được tổ chức theo kiến trúc phi quan hệ phẳng, định dạng cấu trúc tệp văn bản JSON mã hóa UTF-8 để hỗ trợ đầy đủ ký tự Tiếng Việt có dấu. Để thực hiện ánh xạ chính xác cấu trúc kế thừa đa hình của thực thể Reader xuống tệp văn bản tĩnh, hệ thống thiết lập trường định danh lớp con mang tên "type" ngay bên trong cấu trúc JSON.

```
@startjson  "type": "Sinh viên ưu tiên", "userId": "SV001", "fullName": "Nguyễn Văn A",  
            "phoneNumber": "0987654321", "maxBorrowLimit": 5, "finePaid": 0  @endjson
```

Hình 3.2: Cấu trúc ánh xạ tệp tin cứng `readers.json` phục vụ Deserialize đa hình

Khi luồng dữ liệu chạy qua bộ Jackson ObjectMapper, hệ thống tự động đối chiếu giá trị của trường "type" với các cấu hình định danh phân lớp để ánh xạ đối tượng chính xác [11]. Nếu giá trị là "Sinh viên ưu tiên", hệ thống lập tức khởi tạo vùng nhớ kiểu `PriorityStudentReader` trên bộ nhớ RAM, đồng thời tự động nạp chiến lược tính phạt tương ứng là `PriorityStudentFinePolicy` vào biến tham chiếu nội tại của thực thể [11, 8].

Quy trình tuần tự hóa ngược lại (Serialize) khi lưu file cũng diễn ra tương tự: mọi biến động trạng thái đối tượng trên bộ nhớ RAM của hệ thống Java Collections Framework lập tức được chuyển đổi thành chuỗi văn bản phẳng và ghi đè đồng bộ xuống ổ cứng, bảo đảm tính nhất quán dữ liệu liên kết tuyệt đối giữa các tệp `books.json` và `borrow_tickets.json` mà không xảy ra hiện tượng mất mát thông tin [11].

3.4 Cấu Trúc Thư Mục Dự Án

```

HeThongQuanLyThuVien/
- data/ [Thư mục lưu tệp JSON Database]
- src/main/java/com/library/ [Mã nguồn Backend Java Spring Boot]
  - config/ - Cấu hình hệ thống (SecurityConfig)
  - controller/ - Tầng tiếp nhận API (Handler)
  - dto/ - Đối tượng đóng gói dữ liệu
  - exception/ - Bộ Custom Exceptions
  - model/ - Lớp thực thể (Reader, Book, Ticket,...)
  - repository/ - Tầng truy xuất và lưu trữ dữ liệu (Đọc/ghi file JSON)
  - service/ - Tầng xử lý logic nghiệp vụ
- frontend/ [Mã nguồn Frontend React SPA]
  - src/components/ - Các Form nhập liệu nghiệp vụ
  - src/App.jsx - Giao diện điều phối và bảng giám sát
  - vite.config.js - Cấu hình Bundler Vite
- .gitignore [Bộ lọc chặn file rác node_modules]
- pom.xml [Tệp cấu hình dependency Maven]

```

Hình 3.3: Giao diện phân hệ Quản lý thông tin và số lượng tồn kho Sách

CHƯƠNG 4

TRIỂN KHAI CÀI ĐẶT VÀ MÃ NGUỒN CỐT LÕI

4.1 Mã nguồn Tầng Mô hình (Model Layer)

Tầng mô hình (Model Layer) đóng vai trò định hình cấu trúc dữ liệu nền tảng cho toàn bộ hệ thống, phản ánh trực tiếp các thực thể trong không gian bài toán quản lý thư viện vào các đối tượng lập trình Java [5]. Các lớp đối tượng trong tầng này được thiết kế theo tiêu chuẩn mã nguồn sạch, áp dụng nghiêm ngặt nguyên lý đóng gói (Encapsulation), lập trình phòng vệ (Defensive Programming) để bảo vệ toàn vẹn dữ liệu, và nhúng cấu hình ánh xạ đa hình để tương thích hoàn hảo với cơ chế giải tuần tự hóa của Jackson [11, 13].

4.1.1 Cấu trúc phân cấp kế thừa và Đóng gói thực thể người dùng (User, Reader)

Hệ thống thiết lập một cấu trúc phân cấp thực thể người dùng từ trừu tượng đến cụ thể nhằm tối ưu hóa khả năng tái sử dụng mã nguồn và quản lý đa hình động tại thời điểm thực thi (Runtime) [13].

Lớp trừu tượng nền móng User.java áp dụng nghiêm ngặt tính đóng gói bằng cách thiết lập phạm vi truy cập private cho tất cả các trường dữ liệu cốt lõi, bắt buộc mọi tương tác từ bên ngoài phải đi qua hệ thống hàm Getter/Setter công khai kèm cơ chế chuẩn hóa chuỗi dữ liệu tự động [13]:

```
1 package com.library.model;
2
3 public abstract class User {
4     private String userId;
5     private String fullName;
6     private String phoneNumber;
7
8     public User(String userId, String fullName, String phoneNumber) {
9         setUserId(userId);
10        setFullName(fullName);
11        setPhoneNumber(phoneNumber);
12    }
13
14    public String getUserId() { return userId; }
```

```

15
16 public void setUserId(String userId) {
17     if (userId == null || userId.trim().isEmpty()) {
18         throw new IllegalArgumentException("Loi du lieu: Ma nguoi dung khong
19         duoc phep de trong!");
20     }
21     // Tu dong chuan hoa cat khoang trang va viet hoa toan bo ma dinh danh
22     this.userId = userId.trim().toUpperCase();
23 }
24
25 public String getFullName() { return fullName; }
26
27 public void setFullName(String fullName) {
28     if (fullName == null || fullName.trim().isEmpty()) {
29         throw new IllegalArgumentException("Loi du lieu: Ho va ten khong duoc
30         phep de trong!");
31     }
32     this.fullName = fullName.trim();
33 }
34
35 public String getPhoneNumber() { return phoneNumber; }
36
37 public void setPhoneNumber(String phoneNumber) {
38     this.phoneNumber = phoneNumber != null ? phoneNumber.trim() : "";
39 }

```

Listing 4.1: Mã nguồn đóng gói và chuẩn hóa dữ liệu tại User.java

Phân tích giải pháp kỹ thuật tại User.java: Hàm setUserId và setFullName được cài đặt bẫy logic kiểm tra dữ liệu đầu vào (IllegalArgumentException) [5]. Điều này ngăn chặn việc khởi tạo các đối tượng rác hoặc lỗi cấu trúc ngay từ cổng vào của ứng dụng. Việc tích hợp lệnh trim().toUpperCase() bảo đảm tính nhất quán dữ liệu liên kết, loại bỏ hoàn toàn các sai lệch do người dùng nhập chữ thường hoặc thừa dấu cách vật lý khi hệ thống thực hiện tác vụ tìm kiếm bản ghi sau này [13].

Kế thừa từ User, lớp trừu tượng Reader.java đóng vai trò cấu hình hạ tầng metadata đa hình cho bộ tuần tự hóa Jackson ObjectMapper, hỗ trợ lưu trữ dữ liệu phân loại xuống tệp JSON tĩnh nhưng khi nạp lên bộ nhớ RAM vẫn khôi phục chính xác kiểu dữ liệu của lớp con thực tế [11]:

```

1 package com.library.model;
2
3 import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
4 import com.fasterxml.jackson.annotation.JsonIgnore;
5 import com.fasterxml.jackson.annotation.JsonTypeInfo;
6 import com.fasterxml.jackson.annotation.JsonSubTypes;
7 import com.library.service.policy.FinePolicy;

```



```

8
9 @JsonTypeInfo(
10     use = JsonTypeInfo.Id.NAME,
11     include = JsonTypeInfo.As.PROPERTY,
12     property = "readerType" // Khoa anh xa dinh danh phan loai trong tep JSON
13 )
14 @JsonSubTypes({
15     @JsonSubTypes.Type(value = StudentReader.class, name = "Sinh vien thuong")
16     ,
17     @JsonSubTypes.Type(value = PriorityStudentReader.class, name = "Sinh vien
18     uu tien"),
19     @JsonSubTypes.Type(value = LecturerReader.class, name = "Giang vien")
20 })
21 @JsonIgnoreProperties(ignoreUnknown = true)
22 public abstract class Reader extends User {
23
24     @JsonIgnore
25     private final FinePolicy finePolicy; // Ngan chan Jackson serialize giao dien
26     truu tuong
27
28     public Reader(String userId, String fullName, String phoneNumber, FinePolicy
29     finePolicy) {
30         super(userId, fullName, phoneNumber);
31         if (finePolicy == null) {
32             throw new IllegalArgumentException("Loi he thong: Chien luoc tinh phi
33             phat khong duoc phep null!");
34         }
35         this.finePolicy = finePolicy;
36     }
37
38     public FinePolicy getFinePolicy() { return finePolicy; }
39
40     // Cac phuong thuc truu tuong buoc cac lop con phai hien thuc hoa
41     public abstract int getMaxBorrowLimit();
42     public abstract String getReaderType();
43     public abstract int getFinePerDay();
44 }

```

Listing 4.2: Khai báo metadata cấu hình đa hình tại Reader.java

Phân tích giải pháp kỹ thuật tại Reader.java: Hệ thống sử dụng tổ hợp annotation `@JsonTypeInfo` và `@JsonSubTypes` nhằm giải quyết bài toán "mất mát kiểu dữ liệu" (Type Loss) kinh điển khi làm việc với lưu trữ tệp tin cứng phi quan hệ [11]. Thuộc tính `readerType` trong chuỗi JSON đóng vai trò bộ nhãn định danh phân biệt lớp con.

Annotation `@JsonIgnore` đặt trên trường `finePolicy` có ý nghĩa kỹ thuật vô cùng quan trọng: interface `FinePolicy` đại diện cho hành vi xử lý (Mẫu thiết kế Chiến lược - Strategy Pattern) [6, 8]. Nếu không sử dụng `@JsonIgnore`, Jackson sẽ cố gắng chuyển đổi trường này thành cấu trúc JSON độc lập, dẫn đến hiện tượng mã nguồn sinh ra các trường rỗng thừa thãi hoặc gây lỗi sập luồng biên dịch dữ liệu hệ thống [11].

4.1.2 Hiện thực hóa đa hình các nhóm độc giả đặc thù (StudentReader, PriorityStudentReader, LecturerReader)

Ba phân lớp cụ thể đại diện cho các thực thể bạn đọc trong thế giới thực tiến hành hiện thực hóa lớp trừu tượng `Reader`, đồng thời tự động tiêm (Inject) chính xác đối tượng chiến lược tài chính tương ứng vào biến tham chiếu của lớp cha khi được khởi tạo [13, 8].

```
1 package com.library.model;
2
3 import com.library.service.policy.NormalStudentFinePolicy;
4
5 public class StudentReader extends Reader {
6     private static final int    MAX_BORROW_LIMIT = 3;
7     private static final int    FINE_PER_DAY     = 5000;
8     private static final String READER_TYPE      = "Sinh vien thuong";
9
10    public StudentReader() {
11        super("TEMP_ID", "TEMP_NAME", "TEMP_PHONE", new NormalStudentFinePolicy());
12    }
13
14    public StudentReader(String userId, String fullName, String phoneNumber) {
15        super(userId, fullName, phoneNumber, new NormalStudentFinePolicy());
16    }
17
18    @Override
19    public int getMaxBorrowLimit() { return MAX_BORROW_LIMIT; }
20
21    @Override
22    public String getReaderType() { return READER_TYPE; }
23
24    @Override
25    public int getFinePerDay() { return FINE_PER_DAY; }
26 }
```

Listing 4.3: Triển khai logic nghiệp vụ đặc thù tại `StudentReader.java`

Lớp `PriorityStudentReader.java` và `LecturerReader.java` được cấu trúc hóa theo mô hình tương tự,

cô lập các thông số nghiệp vụ đóng gói biệt lập [5]:

- `PriorityStudentReader`: Thiết lập hằng số giới hạn `MAX_BORROW_LIMIT = 5`, định mức hiển thị tra cứu `FINE_PER_DAY = 3000` và tiêm vào vùng nhớ đối tượng thực thi chiến lược `PriorityStudentFinePolicy`.
- `LecturerReader`: Thiết lập quyền lợi cao nhất cho cán bộ giảng viên với hạn mức `MAX_BORROW_LIMIT = 10`, định mức phạt `FINE_PER_DAY = 2000` và liên kết với đối tượng chiến lược `LecturerFinePolicy`.

Phân tích giải pháp kỹ thuật nhóm lớp con `Reader`: Sự xuất hiện của các constructor mặc định không tham số (`public StudentReader()`) là yêu cầu bắt buộc kỹ thuật đối với cơ chế giải tuần tự hóa của thư viện Jackson [11]. Khi ánh xạ file JSON ngược lên bộ nhớ RAM, Jackson sử dụng cơ chế phản chiếu mã nguồn (Reflection API) để gọi constructor không tham số nhằm khởi dựng thực thể thô trước, sau đó mới dùng các hàm Setter hoặc trường truy cập để đổ dữ liệu vào thực thể đối tượng [13, 11]. Việc gọi các giá trị khởi tạo giả lập tạm thời như `"TEMP_ID"` đảm bảo lớp cha `Reader` luôn nhận được cấu trúc an toàn không bị lỗi con trỏ rỗng (`NullPointerException`).

4.1.3 Thiết kế cấu trúc phức hợp và Đối tượng bất biến (`BorrowTicket`, `BorrowTicketDetail`, `Book`)

Đối với các thực thể mang thuộc tính quản lý tài sản và lưu vết lịch sử giao dịch trực tiếp của hệ thống thư viện, mã nguồn áp dụng các kỹ thuật phòng vệ bộ nhớ nâng cao để tránh hiện tượng mất mát thông tin hoặc thay đổi trạng thái đối tượng ngoài ý muốn [5].

Lớp `Book.java` định nghĩa đầy đủ 6 thuộc tính kho tài nguyên, tích hợp các bẫy chặn toán học trong hàm Setter để triệt tiêu hoàn toàn nguy cơ gán giá trị âm cho số lượng tồn kho hoặc giá thành vật lý của sách:

```
1 package com.library.model;
2
3 public class Book {
4     private String bookId;
5     private String title;
6     private String author;
7     private String category;
8     private int quantity;
9     private double price;
10
11     public Book() {}
12
13     public Book(String bookId, String title, String author, String category, int
quantity, double price) {
14         this.bookId = bookId != null ? bookId.trim() : "";
15         this.title = title != null ? title.trim() : "";
```

```

16     this.author = author != null ? author.trim() : "Chua ro";
17     this.category = category != null ? category.trim() : "Giao trinh";
18     setQuantity(quantity);
19     setPrice(price);
20 }
21
22 public String getBookId() { return bookId; }
23 public void setBookId(String bookId) { this.bookId = bookId != null ? bookId.
trim() : ""; }
24 public String getTitle() { return title; }
25 public void setTitle(String title) { this.title = title != null ? title.trim()
: ""; }
26
27 public int getQuantity() { return quantity; }
28 public void setQuantity(int quantity) {
29     // Chan gia tri am de bao du lieu kho dong bo
30     this.quantity = Math.max(0, quantity);
31 }
32
33 public double getPrice() { return price; }
34 public void setPrice(double price) {
35     // Gia tri tai san khong duoc phep nho hon 0.0
36     this.price = Math.max(0.0, price);
37 }
38 }

```

Listing 4.4: Mã nguồn bảo vệ toàn vẹn dữ liệu kho tại Book.java

Lớp BorrowTicketDetail.java được thiết kế theo mô hình đối tượng bất biến (Immutable Object) bằng cách khai báo trường thuộc tính ở phạm vi private final và triệt tiêu hoàn toàn các hàm Setter sửa đổi dữ liệu [5]. Nhằm giúp Jackson có thể đổ dữ liệu trực tiếp vào một đối tượng Immutable, hệ thống nhúng annotation @JsonCreator điều phối luồng ánh xạ chui thẳng vào tham số của hàm khởi tạo:

```

1 package com.library.model;
2
3 import com.fasterxml.jackson.annotation.JsonCreator;
4 import com.fasterxml.jackson.annotation.JsonProperty;
5 import java.util.Objects;
6
7 public final class BorrowTicketDetail {
8     private final String bookId;
9     private final int quantity;
10
11     @JsonCreator
12     public BorrowTicketDetail(
13         @JsonProperty("bookId") String bookId,

```

```

14         @JsonProperty("quantity") int quantity) {
15         if (bookId == null || bookId.trim().isEmpty()) {
16             throw new IllegalArgumentException("Loi: Ma sach chi tiet khong duoc
rong!");
17         }
18         if (quantity <= 0) {
19             throw new IllegalArgumentException("Loi: So luong sach muon phai lon
hon 0!");
20         }
21         this.bookId = bookId.trim().toUpperCase();
22         this.quantity = quantity;
23     }

24
25     public String getBookId() { return bookId; }
26     public int getQuantity() { return quantity; }
27
28     @Override
29     public boolean equals(Object o) {
30         if (this == o) return true;
31         if (o == null || getClass() != o.getClass()) return false;
32         BorrowTicketDetail that = (BorrowTicketDetail) o;
33         return quantity == that.quantity && Objects.equals(bookId, that.bookId);
34     }
35
36     @Override
37     public int hashCode() {
38         return Objects.hash(bookId, quantity);
39     }
40 }

```

Listing 4.5: Hiện thực hóa đối tượng bất biến tại BorrowTicketDetail.java

Phân tích giải pháp kỹ thuật tại BorrowTicketDetail.java: Do thực thể chi tiết phiếu mượn mang tính chất là một bản ghi lịch sử, việc áp dụng đặc tính final bảo đảm dữ liệu sau khi ghi nhận sẽ đóng băng vĩnh viễn trên RAM, loại bỏ hoàn toàn bấy lỗi các module khác sửa đổi nhầm số lượng mượn [5]. Việc ghi đè song hành hai phương thức equals() và hashCode() tuân thủ nghiêm ngặt giao kèo thiết kế hướng đối tượng [13]: nếu không hiện thực hóa hashCode(), hệ thống Java Collections Framework sẽ không thể nhận diện hai chi tiết phiếu trùng nhau khi lưu trữ vào cấu trúc bảng băm, gây lỗi nghiêm trọng khi chạy thuật toán dọn dẹp hoặc đối chiếu phần tử mảng.

Lớp quản lý phức hợp trung tâm BorrowTicket.java tập hợp danh sách các chi tiết phiếu mượn lồng nhau, áp dụng kỹ thuật sao chép phòng vệ bảo mật (Defensive Copy) để ngăn chặn rò rỉ vùng nhớ [5]:

```

1 package com.library.model;
2

```

```

3 import java.time.LocalDate;
4 import java.util.ArrayList;
5 import java.util.Collections;
6 import java.util.List;
7
8 public class BorrowTicket {
9     public enum TicketStatus {
10         BORROWING("Dang muon"), RETURNED("Da tra");
11         private final String value;
12         TicketStatus(String value) { this.value = value; }
13         public String getValue() { return value; }
14     }
15
16     private String ticketId;
17     private String readerId;
18     private LocalDate borrowDate;
19     private LocalDate dueDate;
20     private LocalDate returnDate;
21     private TicketStatus status;
22     private long finePaid;
23     private List<BorrowTicketDetail> details = new ArrayList<>();
24
25     public List<BorrowTicketDetail> getDetails() {
26         // Tra ve mot ban sao bo nho Chi doc de bao ve an toan danh sach goc tren
27         // RAM
28         return Collections.unmodifiableList(details);
29     }
30
31     public void setDetails(List<BorrowTicketDetail> details) {
32         this.details = details != null ? new ArrayList<>(details) : new ArrayList<>();
33     }
34 }

```

Listing 4.6: Kỹ thuật sao chép phòng vệ tại BorrowTicket.java

Phân tích giải pháp kỹ thuật tại BorrowTicket.java: Hàm getDetails() không trả về trực tiếp con trỏ vùng nhớ của biến details. Thay vào đó, nó bọc danh sách lại bằng phương thức Collections.unmodifiableList() [13]. Kỹ thuật này phát ra một cam kết bảo mật: nếu bất kỳ tầng mã nguồn nào phía trên cố tình gọi lệnh .add() hoặc .remove() trên danh sách chi tiết này, hệ thống Java Runtime lập tức ném ra lỗi chặn đứng UnsupportedOperationException, cô lập hoàn toàn trạng thái an toàn của phiếu mượn gốc [5].

4.1.4 Mô hình phẳng hóa dữ liệu giao tiếp mặt trước (BorrowRecord, ReturnRecord)

Để giải quyết bài toán tối ưu băng thông đường truyền và tốc độ kết xuất đồ họa đồng bộ của máy trạm React Single Page Application, hệ thống triển khai phân lớp phẳng hóa dữ liệu giao tiếp đóng vai trò như các thực thể vận chuyển thông tin tinh gọn [7, 9].

Lớp BorrowRecord.java thực hiện cấu trúc phẳng hóa dữ liệu đa tầng từ phiếu mượn phức hợp, chuyển đổi các đối tượng thời gian phức tạp thành chuỗi văn bản mạch lạc, cung cấp hàm khởi tạo 5 tham số đồng bộ hoàn hảo với cột hiển thị ngày hạn trả sách (dueDate) tại giao diện bảng:

```
1 package com.library.model;
2
3 public class BorrowRecord {
4     private String readerId;
5     private String bookId;
6     private int quantity;
7     private String borrowDate;
8     private String dueDate; // Truong du lieu moi phuc vu giam sat Realtime
9
10    public BorrowRecord() {}
11
12    public BorrowRecord(String readerId, String bookId, int quantity, String
13    borrowDate, String dueDate) {
14        this.readerId = readerId;
15        this.bookId = bookId;
16        this.quantity = quantity;
17        this.borrowDate = borrowDate;
18        this.dueDate = dueDate;
19    }
20
21    public String getReaderId() { return readerId; }
22    public void setReaderId(String readerId) { this.readerId = readerId; }
23    public String getBookId() { return bookId; }
24    public void setBookId(String bookId) { this.bookId = bookId; }
25    public int getQuantity() { return quantity; }
26    public void setQuantity(int quantity) { this.quantity = quantity; }
27    public String getBorrowDate() { return borrowDate; }
28    public void setBorrowDate(String borrowDate) { this.borrowDate = borrowDate; }
29    public String getDueDate() { return dueDate; }
30    public void setDueDate(String dueDate) { this.dueDate = dueDate; }
31 }
```

Listing 4.7: Mã nguồn phẳng hóa dữ liệu truyền tải tại BorrowRecord.java

Song hành với đó, lớp ReturnRecord.java quản lý luồng nhật ký thu hồi tài nguyên và quyết toán tài chính, sử dụng kiểu dữ liệu số nguyên lớn long cho trường thuộc tính finePaid nhằm triệt tiêu hoàn

toàn hiện tượng sai lệch dấu phẩy động hoặc làm tròn số tự động thường gặp của kiểu dữ liệu thập phân khi xử lý các giao dịch tài chính thư viện [5].

```
1 package com.library.model;
2
3 import java.time.LocalDate;
4
5 public class ReturnRecord {
6     private String recordId;
7     private String ticketId;
8     private LocalDate returnDate;
9     private long finePaid;           // Chuyển đổi sang long để tránh tiêu sai số dấu
    phẩy động
10
11     public ReturnRecord() {}
12
13     public ReturnRecord(String recordId, String ticketId, LocalDate returnDate,
14 long finePaid) {
15         this.recordId = recordId != null ? recordId.trim() : "";
16         this.ticketId = ticketId != null ? ticketId.trim() : "";
17         this.returnDate = returnDate != null ? returnDate : LocalDate.now();
18         setFinePaid(finePaid);
19     }
20
21     public String getRecordId() { return recordId; }
22     public String getTicketId() { return ticketId; }
23     public LocalDate getReturnDate() { return returnDate; }
24
25     public long getFinePaid() { return finePaid; }
26     public void setFinePaid(long finePaid) {
27         this.finePaid = Math.max(0L, finePaid);
28     }
29 }
```

Listing 4.8: Mã nguồn lưu nhật ký quyết toán tài chính tại ReturnRecord.java

4.2 Mã nguồn Tầng Nghiệp vụ (Service Layer)

Tầng nghiệp vụ (Service Layer) đóng vai trò là bộ não điều khiển trung tâm của toàn bộ hệ thống, chịu trách nhiệm cô lập logic xử lý quy trình, thực thi các quy tắc ràng buộc, phối hợp các thao tác lưu trữ dữ liệu và xử lý các bài toán tài chính phức tạp [7]. Được đánh dấu bằng annotation `@Service` của Spring Framework, các lớp dịch vụ tại phân tầng này nhận dữ liệu thô hoặc các đối tượng DTO từ tầng Controller, bóc tách cấu trúc phức hợp, điều phối hạ tầng Repository và đảm bảo tính toàn vẹn trạng thái của toàn bộ hệ thống trước khi phản hồi về phía máy trạm [10, 5].

4.2.1 Logic điều khiển Lưu thông và Quyết toán đa hình (BorrowReturnService.java)

`BorrowReturnService.java` là thành phần kiến trúc cốt lõi nhất của dự án, nơi trực tiếp triển khai các thuật toán phẳng hóa cấu trúc dữ liệu đa tầng phục vụ đồ họa React, kiểm soát hạn mức mượn và áp dụng mẫu thiết kế Chiến lược (Strategy Pattern) để quyết toán tiền phạt quá hạn [6, 8].

Phương thức `getAllRecords()` giải quyết bài toán biến đổi cấu trúc dữ liệu lồng nhau (Nested data) từ tệp cứng `borrow_tickets.json` thành một danh sách mảng một chiều phẳng (`List<BorrowRecord>`), tương thích trực tiếp với cấu trúc hiển thị của thành phần bảng theo dõi lưu thông thời gian thực ở phía máy trạm React [9]:

```
1 package com.library.service;
2
3 import com.library.model.*;
4 import java.time.LocalDate;
5 import java.time.temporal.ChronoUnit;
6 import java.util.ArrayList;
7 import java.util.List;
8
9 public class BorrowReturnService {
10     // ... [Cac bien Repository va co che Dependency Injection]
11
12     public List<BorrowRecord> getAllRecords() {
13         List<BorrowRecord> activeRecords = new ArrayList<>();
14         List<BorrowTicket> allTickets = borrowTicketRepository.getAll();
15
16         // Vong la bock tach va phang hoa cau truc du lieu da tang
17         for (BorrowTicket t : allTickets) {
18             if (t.getStatus() == BorrowTicket.TicketStatus.BORROWING) {
19                 for (BorrowTicketDetail d : t.getDetails()) {
20                     // Anh xa phang hoa thong tin sang doi tuong tinh gon
21                     BorrowRecord rec = new BorrowRecord(
22                         t.getReaderId(),
23                         d.getBookId(),
```

```

24         d.getQuantity() ,
25         t.getBorrowDate().toString() ,
26         t.getDueDate().toString()
27     );
28     activeRecords.add(rec);
29 }
30 }
31 }
32 return activeRecords;
33 }
34 }

```

Listing 4.9: Thao tác phẳng hóa và bóc tách mảng lồng nhau tại BorrowReturnService.java

Phân tích giải pháp kỹ thuật phẳng hóa dữ liệu: Về mặt cấu trúc lưu trữ, một phiếu mượn (BorrowTicket) chứa thông tin chung của bạn đọc và một danh sách các chi tiết sách mượn song hành (List<BorrowTicketDetail>) [13]. Thiết kế này tối ưu cho việc lưu file JSON độc lập nhưng lại gây khó khăn cho việc hiển thị dạng bảng lưới phẳng (Grid/Table view) trên giao diện Frontend. Thuật toán duyệt lồng nhau phía trên bóc tách cấu trúc phức hợp này thành các bản ghi BorrowRecord độc lập tuyến tính, giúp giảm tải toàn bộ áp lực tính toán xử lý chuỗi cho trình duyệt Client, duy trì kiến trúc Decoupled chuẩn mực [3].

4.2.2 Kiểm tra ràng buộc nghiệp vụ và Quản lý danh mục Thực thể (BookService, ReaderService)

Song hành với phân hệ lưu thông, BookService.java và ReaderService.java đảm nhiệm vai trò thực thi các quy tắc xác thực an toàn thông tin đầu vào (Data Validation) và bẫy chặn các hành vi vi phạm ràng buộc liên kết dữ liệu trước khi chuyển giao tác vụ xuống tầng lưu trữ tệp tin cứng [5].

Lớp ReaderService.java triển khai cơ chế kiểm tra định dạng dữ liệu nghiêm ngặt đối với hồ sơ đọc giả bằng biểu thức chính quy (Regex) và bẫy lỗi tùy biến tập trung [13]:

```

1 package com.library.service;
2
3 import com.library.exception.DataNotFoundException;
4 import com.library.model.Reader;
5 import com.library.repository.ReaderRepository;
6 import java.util.List;
7
8 public class ReaderService {
9     private final ReaderRepository readerRepository;
10
11     public ReaderService(ReaderRepository readerRepository) {
12         this.readerRepository = readerRepository;
13     }
14 }

```

```

15     public void registerReader(Reader reader) {
16         if (reader == null) {
17             throw new IllegalArgumentException("Loi: Du lieu doc gia khoi tao
18             khong duoc null!");
19         }
20         if (readerRepository.findById(reader.getUserId()) != null) {
21             throw new IllegalArgumentException("Loi nhiep vu: Ma doc gia '" +
22             reader.getUserId() + "' da ton tai!");
23         }
24         validateReaderData(reader.getFullName(), reader.getPhoneNumber());
25         readerRepository.addReader(reader);
26     }
27
28     private void validateReaderData(String fullName, String phoneNumber) {
29         if (fullName == null || fullName.trim().isEmpty()) {
30             throw new IllegalArgumentException("Loi nhiep vu: Ho va ten ban doc
31             khong duoc de trong!");
32         }
33         if (fullName.trim().length() < 2) {
34             throw new IllegalArgumentException("Loi nhiep vu: Ho va ten qua ngan!
35             ");
36         }
37         if (phoneNumber == null || phoneNumber.trim().isEmpty()) {
38             throw new IllegalArgumentException("Loi nhiep vu: So dien thoai lien
39             lac khong duoc de trong!");
40         }
41         // Kiem tra dinh dang so dien thoai phai la chuoi so dai tu 10 den 11 ky
42         tu
43         if (!phoneNumber.trim().matches("\\\\d{10,11}")) {
44             throw new IllegalArgumentException("Loi dinh dang: So dien thoai phai
45             gom 10 hoac 11 chu so vat ly!");
46         }
47     }
48 }

```

Listing 4.10: Quy trình xác thực dữ liệu và quản lý ban đọc tại ReaderService.java

Lớp BookService.java kiểm soát toàn bộ vòng đời của kho tài nguyên sách, ứng dụng bộ custom checked exception DataNotFoundException để tăng cường tính minh bạch của thông điệp hệ thống khi tương tác với các tệp tin cứng JSON [5]:

```

1 package com.library.service;
2
3 import com.library.exception.DataNotFoundException;
4 import com.library.model.Book;
5 import com.library.repository.BookRepository;

```

```

6
7 public class BookService {
8     private final BookRepository bookRepository;
9
10    public BookService(BookRepository bookRepository) {
11        this.bookRepository = bookRepository;
12    }
13
14    public Book getBookById(String bookId) throws DataNotFoundException {
15        if (bookId == null || bookId.trim().isEmpty()) {
16            throw new IllegalArgumentException("Loi: Ma sach tim kiem khong duoc
17            de trong!");
18        }
19
20        Book book = bookRepository.findById(bookId.trim());
21        if (book == null) {
22            throw new DataNotFoundException("Loi nghiệp vụ: Không tìm thấy đầu
23            sach mang ma '" + bookId + "' trong hệ thống thư viện!");
24        }
25        return book;
26    }
27
28    public void deleteBook(String bookId) throws DataNotFoundException {
29        String cleanId = bookId != null ? bookId.trim() : "";
30        if (bookRepository.findById(cleanId) == null) {
31            throw new DataNotFoundException("Loi: Không tìm thấy đầu sách mang ma
32            '" + bookId + "' trong kho cùng JSON để xóa!");
33        }
34        bookRepository.deleteBook(cleanId);
35    }
36 }

```

Listing 4.11: Kỹ thuật bắt lỗi danh mục hệ thống tại BookService.java

Phân tích giải thuật kiểm tra ràng buộc danh mục: Toàn bộ mã nguồn trong ReaderService và BookService thể hiện sâu sắc tư duy thiết kế phòng vệ (Defensive Programming) [5]. Việc sử dụng hàm matches("

d10,11") chặn đứng hoàn toàn nguy cơ người dùng nhập sai ký tự chữ vào trường số điện thoại trước khi lưu trữ xuống cơ sở dữ liệu tệp tin [13].

Đồng thời, việc chủ động ném ra custom exception DataNotFoundException thay vì trả về giá trị null giúp phân tách rõ ràng giữa hai trạng thái hệ thống: trạng thái vận hành bình thường và trạng thái lỗi nghiệp vụ, tạo điều kiện thuận lợi cho tầng định tuyến Controller đánh chặn lỗi tập trung và chuyển đổi thông điệp cảnh báo hiển thị trực quan lên màn hình giao diện Dashboard phía Client [10].

4.3 Mã nguồn Xử lý Hoàn trả sách và Quyết toán tài chính

Phân hệ xử lý hoàn trả sách và quyết toán tài chính khi độc giả phát sinh nợ phạt trễ hạn là một trong những phân hệ nghiệp vụ phức tạp nhất của hệ thống. Luồng xử lý này đòi hỏi sự phối hợp chặt chẽ giữa việc thay đổi trạng thái bản ghi lịch sử, cập nhật dữ liệu tồn kho vật lý và kích hoạt cơ chế đa hình động để tính toán chi phí phạt dựa trên mẫu thiết kế Chiến lược (Strategy Pattern) [6, 8]. Toàn bộ logic lõi của quy trình này được bọc gọn trong phương thức `returnBook()` thuộc lớp `BorrowReturnService.java` [7].

Dưới đây là mã nguồn cài đặt chi tiết của phương thức xử lý hoàn trả và quyết toán tài chính với hệ thống chú thích đã được chuẩn hóa không dấu bảo đảm an toàn biên dịch [13]:

```
1  public long returnBook(String readerId , String bookId) throws
   DataNotFoundException {
2      String cleanReaderId = readerId != null ? readerId.trim() : "";
3      String cleanBookId = bookId != null ? bookId.trim().toUpperCase() : "";
4
5      // 1. Quét toàn bộ danh sách để tìm kiếm phiếu mượn đang hoạt động (
BORROWING)
6      List<BorrowTicket> allTickets = borrowTicketRepository.getAll();
7      BorrowTicket targetTicket = null;
8
9      for (BorrowTicket t : allTickets) {
10         if (t.getReaderId().equalsIgnoreCase(cleanReaderId)
11             && t.getStatus() == BorrowTicket.TicketStatus.BORROWING) {
12             for (BorrowTicketDetail d : t.getDetails()) {
13                 if (d.getBookId().equalsIgnoreCase(cleanBookId)) {
14                     targetTicket = t;
15                     break;
16                 }
17             }
18         }
19         if (targetTicket != null) break;
20     }
21
22     // Nếu không tìm thấy card Reader - Book nào đang mượn, ném ra lỗi nghiệp
vu
23     if (targetTicket == null) {
24         throw new DataNotFoundException("Lỗi nghiệp vụ: Không tìm thấy giao
dịch đang mượn phù hợp!");
25     }
26
27     Reader reader = readerRepository.findById(cleanReaderId);
28     Book book = bookRepository.findById(cleanBookId);
29
```

```

30 // 2. Hoan tra lai so luong sach vao kho va dong bo xuong file cung JSON
31 if (book != null) {
32     int borrowedQuantity = targetTicket.getDetails().stream()
33         .filter(d -> d.getBookId().equalsIgnoreCase(cleanBookId))
34         .mapToInt(BorrowTicketDetail::getQuantity)
35         .sum();
36     book.setQuantity(book.getQuantity() + borrowedQuantity);
37     bookRepository.update(book); // Luu ghi de lai file books.json
38 }
39
40 // 3. Thiet la ngay tra thuc te va dong trang thai phieu muon
41 LocalDate returnDate = LocalDate.now();
42 targetTicket.setReturnDate(returnDate);
43 targetTicket.setStatus(BorrowTicket.TicketStatus.RETURNED);
44
45 // Tinh toan chinh xac khoang cach so ngay tre han bang thu vien Java Time
46 // API
47 long overdueDays = ChronoUnit.DAYS.between(targetTicket.getDueDate(),
48     returnDate);
49 long fineAmount = 0L;
50
51 // 4. Neu vuot qua han tra, kich hoat da hinh Strategy Pattern de quyét
52 // toan tai chinh
53 if (overdueDays > 0) {
54     fineAmount = (long) reader.getFinePolicy().calculateFine((int)
55     overdueDays);
56     targetTicket.setFinePaid(fineAmount);
57 }
58
59 // Cap nhat lai trang thai phieu va ghi de dong bo xuong file
60 borrow_tickets.json
61 borrowTicketRepository.update(targetTicket);
62 return fineAmount;
63 }

```

Listing 4.12: Phương thức xử lý thu hồi sách và quyết toán tài chính tại BorrowReturnService.java

Phân tích giải pháp kỹ thuật quyết toán tài chính: Hàm returnBook() thể hiện tư duy lập trình hướng đối tượng kết hợp phòng vệ bộ nhớ sâu sắc qua việc tính toán khoảng cách ngày quá hạn dựa vào câu lệnh `ChronoUnit.DAYS.between(targetTicket.getDueDate(), returnDate)` loại bỏ hoàn toàn các sai số tính toán thủ công về năm nhuận hoặc số ngày lệch giữa các tháng trong năm [13].

Đặc biệt, tại dòng lệnh `reader.getFinePolicy().calculateFine((int) overdueDays)`, lập trình viên hoàn toàn không sử dụng bất kỳ mệnh đề rẽ nhánh kiểm tra kiểu nào. Interface `FinePolicy` đóng vai trò như một bộ khung trừu tượng [7]. Tại thời điểm chạy (Runtime), máy ảo Java (JVM) dựa vào vùng nhớ

thực tế của thực thể đọc giả được Jackson khôi phục từ tệp JSON (ví dụ: lớp `LecturerReader`) để tự động kích hoạt chiến lược tính tiền phạt tương ứng của lớp đó (`LecturerFinePolicy`), trả về số tiền phạt chính xác kiểu long nhằm triệt tiêu hoàn toàn hiện tượng sai lệch dữ liệu [13, 11].

4.4 Mã nguồn Giao diện Frontend (React.js) - Bảng điều khiển Giám sát

Tầng giao diện (Frontend) của hệ thống được xây dựng dựa trên kiến trúc ứng dụng đơn trang Single Page Application (SPA) sử dụng thư viện React.js [9]. Trong mô hình tách biệt hoàn toàn (Decoupled Architecture), Frontend đóng vai trò là một máy trạm độc lập, chịu trách nhiệm quản lý trạng thái hiển thị cục bộ (Local State Management), điều phối luồng trải nghiệm người dùng và thực hiện gọi các cổng dịch vụ dữ liệu thô (RESTful API) từ Backend một cách hoàn toàn bất đồng bộ thông qua Fetch API [3, 15].

4.4.1 Cơ chế quản lý trạng thái và Gọi API bất đồng bộ (App.jsx)

Trung tâm điều phối giao diện của hệ thống nằm tại tệp tin App.jsx. Tại đây, hệ thống ứng dụng các Hook cốt lõi của React là useState để duy trì mảng dữ liệu động và useEffect để thiết lập các hiệu ứng lề (Side-effects), tự động đồng bộ hóa bảng theo dõi lưu thông sách ngay khi máy trạm vừa khởi chạy [9].

Dưới đây là mã nguồn xử lý logic kết nối mạng, tải danh mục phiếu mượn hiện hành và xử lý tác vụ đóng giao dịch lưu thông siêu tốc với hệ thống chú thích đã được chuẩn hóa không dấu [15]:

```
1 // --- PHAN HE GIAO DICH MUON / TRA SACH (APP.JSX) ---
2 const [borrowList, setBorrowList] = useState([]);
3 const [isLoadingBorrows, setIsLoadingBorrows] = useState(false);
4
5 // 1. API: Doc toan bo danh sach phieu muon hoat dong tu Backend
6 const fetchBorrows = async () => {
7     setIsLoadingBorrows(true);
8     try {
9         const res = await fetch('http://localhost:8080/api/v1/borrow/records');
10        if (res.ok) {
11            const data = await res.json();
12            setBorrowList(data); // Cap nhat mang phang vao State tren RAM Client
13        } else {
14            console.error("Loi phan hoi tu server khi doc danh sach muon");
15        }
16    } catch (err) {
17        console.error("Loi ket noi API he thong:", err);
18    } finally {
19        setIsLoadingBorrows(false);
20    }
21 };
22
23 // Auto trigger ham doc du lieu khi component duoc nap lan dau
24 useEffect(() => {
25     fetchBorrows();
26 }, []);
```



```

27
28 // 2. API: Xu ly nghiệp vụ "Tra nhanh" trực tiếp từ dữ liệu của bảng
29 const handleQuickReturn = async (readerId, bookId) => {
30     if (!window.confirm('Xác nhận thu hồi đầu sách [{bookId}] của độc giả [{readerId}]?')) {
31         return;
32     }
33     try {
34         const res = await fetch('http://localhost:8080/api/v1/borrow/return', {
35             method: 'POST',
36             headers: { 'Content-Type': 'application/json' },
37             body: JSON.stringify({ readerId, bookId }) // Dong gọi dữ liệu
38             ReturnRequestDTO
39         });
40
41         if (res.ok) {
42             const fineAmount = await res.json();
43             if (fineAmount > 0) {
44                 alert('Tra sách thành công! Độc giả trễ hạn và phải nộp phí phạt: ${fineAmount.toLocaleString()} VND. ');
45             } else {
46                 alert("Tra sách thành công! Giao dịch hợp lệ và không phát sinh phí quá hạn.");
47             }
48             // Dong bỏ lại giao diện ngay lập tức sau khi quyết toán thành công
49             fetchBorrows();
50         } else {
51             const errorText = await res.text();
52             alert('Lỗi nghiệp vụ từ server: ${errorText}');
53         }
54     } catch (err) {
55         alert("Lỗi kết nối API: Không thể thực hiện tác vụ tra nhanh!");
56         console.error(err);
57     }
58 };

```

Listing 4.13: Logic truy xuất API dữ liệu mượn trả bất đồng bộ tại App.jsx

Phân tích giải pháp kỹ thuật logic API máy trạm: Hàm `fetchBorrows()` và `handleQuickReturn()` ứng dụng triệt để cú pháp xử lý bất đồng bộ hiện đại `async/await`, giúp luồng mã nguồn chạy tuyến tính, dễ đọc và tránh được bầy lỗi lồng nhau (Callback Hell) [15]. Phương thức `handleQuickReturn()` đóng vai trò nén thông tin mã bạn đọc và mã sách vào một chuỗi JSON tối giản để khớp nối chính xác với cấu trúc lớp nhận dữ liệu `ReturnRequestDTO` ở Tầng điều hướng Backend.

Điểm đắt giá về mặt trải nghiệm người dùng nằm ở câu lệnh gọi lại `fetchBorrows()` đặt bên trong

khỏi if (res.ok). Ngay sau khi Backend xử lý hoàn kho vật lý và quyết toán tài chính thành công, Frontend lập tức phát lệnh kéo dữ liệu mới về để tái cấu trúc lại mảng trạng thái borrowList. Cơ chế này ép giao diện React tự động vẽ lại bảng giám sát thời gian thực (Reactive Re-rendering) mà không đòi hỏi thủ thư phải nhấn phím F5 làm mới lại toàn bộ trang trình duyệt [9].

4.4.2 Cấu trúc giao diện cấu trúc khối Bảng giám sát Lưu thông Sách

Để kết xuất thông tin trực quan cho thủ thư, hệ thống tổ chức một khối chức năng riêng biệt mang tên Sổ giám sát lưu thông sách, ứng dụng lưới Bootstrap 5 và cấu trúc phẳng hóa dữ liệu BorrowRecord truyền về từ API [9]:

```
1 // activeTab === 'dashboard'
2 borrowList.length === 0 ? (
3   <div>Hien tai khong co phien mun nao dang hoat dong tren he thong thu vien!</div>
4 ) : (
5   <div className="table-responsive">
6     <table className="table table-hover align-middle">
7       <thead className="table-light text-secondary">
8         <tr>
9           <th>Ma Doc Gia</th>
10          <th>Ma Dau Sach</th>
11          <th>So Luong</th>
12          <th>Ngay Muon</th>
13          <th>Han Hoan Tra</th>
14          <th className="text-center">Hanh Dong</th>
15        </tr>
16      </thead>
17      <tbody>
18        {borrowList.map((b, index) => (
19          <tr key={index}>
20            <td className="fw-semibold text-dark">{b.readerId}</td>
21            <td><span className="badge bg-secondary px-2 py-1">{b.
22              bookId}</span></td>
23            <td>{b.quantity || 1} cuon</td>
24            <td className="text-muted">{b.borrowDate}</td>
25            <td className="text-danger fw-bold">{b.dueDate || 'Chua
26              thiet lap'}</td>
27            <td className="text-center">
28              <button
29                className="btn btn-sm btn-outline-warning fw-bold
30                  px-3 py-1 rounded-pill"
31                onClick={() => handleQuickReturn(b.readerId, b.
32                  bookId)}>
```

```

29         >
30         Tra nhanh
31     </button>
32 </td>
33 </tr>
34   )}
35 </tbody>
36 </table>
37 </div>
38 )

```

Listing 4.14: Cấu trúc render giao diện HTML/JSX của bảng điều khiển giám sát

Phân tích giải pháp kỹ thuật cấu trúc JSX: Khối giao diện điều khiển thể hiện tính chất lập trình mô tả chặt chẽ của ngôn ngữ JSX [9]. Biểu thức toán học rút gọn `borrowList.length === 0` (Ternary Operator) giúp hệ thống tự động rẽ nhánh giao diện thông minh: nếu hệ thống sạch bóng dữ liệu, màn hình lập tức hiển thị nhãn thông báo cục bộ; ngược lại, nếu kho dữ liệu chứa bản ghi, cấu trúc bảng HTML sẽ ngay lập tức được dựng lên [15].

Trong khối mã lặp `borrowList.map()`, thuộc tính `key={index}` được nhúng vào cấu trúc để hỗ trợ giải thuật so sánh cây cấu trúc ảo (Virtual DOM Diffing Algorithm) của lõi React [9]. Mỗi dòng bản ghi được gán một chỉ mục duy nhất, cho phép React theo dõi chính xác vị trí dòng nào vừa bị loại bỏ khi thủ thư bấm nút "Trả nhanh". Đặc biệt, cột "Hạn Hoàn Trả" được cấu hình lớp màu sắc `text-danger fw-bold` nổi bật, giúp thủ thư ngay lập tức nhận diện được các độc giả đang có dấu hiệu quá hạn giữ tài sản để nhanh chóng đưa ra các nhắc nhở nghiệp vụ hiệu quả [9].

CHƯƠNG 5

THỬ NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

5.1 Kịch bản kiểm thử (Test Cases)

Để đảm bảo hệ thống vận hành đúng chuẩn theo đặc tả yêu cầu nghiệp vụ và bảo đảm tính toàn vẹn dữ liệu xuyên suốt các phân tầng kiến trúc, nhóm đã tiến hành xây dựng và thực thi các kịch bản kiểm thử hộp đen (Black-box Testing) tập trung vào các bẫy lỗi logic và cơ chế đồng bộ hóa dữ liệu [1, 2]. Chi tiết các kịch bản kiểm thử cốt lõi được mô tả một cách tường minh qua hệ thống bảng biểu dưới đây:

5.1.1 Kịch bản kiểm thử 1: Xác thực tính toàn vẹn định dạng lưu trữ UTF-8

Kịch bản này nhằm xác mục tiêu hệ thống xử lý hoàn hảo dữ liệu Tiếng Việt có dấu, không xảy ra lỗi phân rã chuỗi hoặc lỗi dữ liệu khi Jackson thực hiện serialize xuống ổ đĩa [11].

Mã số Test Case	TC-01: Kiểm tra lưu trữ ký tự Tiếng Việt đa byte xuống JSON
Mục tiêu	Xác thực chức năng khởi tạo tài khoản độc giả chứa ký tự Tiếng Việt có dấu và kiểm tra tính toàn vẹn cấu trúc file cứng readers.json [11].
Dữ liệu đầu vào	Mã bạn đọc: BD099, Họ và tên: Nguyễn Khắc Hoàng Vũ, Số điện thoại: 0912345678, Phân loại: Sinh viên ưu tiên.
Các bước thực hiện	1. Truy cập tab Quản lý độc giả trên giao diện React SPA [9]. 2. Nhập đầy đủ thông tin dữ liệu có dấu vào biểu mẫu (ReaderForm). 3. Nhấn nút "Đăng ký thành viên" để phát request gửi về Server [10]. 4. Mở tệp cứng data/readers.json trên Backend để kiểm tra.
Kết quả kỳ vọng	Backend xử lý thành công, trả về trạng thái HTTP Status 200 OK. Tệp readers.json lưu trữ nguyên vẹn chuỗi "Nguyễn Khắc Hoàng Vũ" theo bảng mã mã hóa chuẩn UTF-8, không bị lỗi hiển thị ký tự lạ [11].
Kết quả thực tế	Hệ thống đồng bộ mượt mà, Frontend cập nhật dòng mới lên bảng ngay lập tức, tệp cứng ghi nhận chuẩn xác dữ liệu UTF-8.
Đánh giá	ĐẠT (PASSED)

Bảng 5.1: Kịch bản kiểm thử chức năng ghi nhận dữ liệu Tiếng Việt đa byte

5.1.2 Kịch bản kiểm thử 2: Bẫy chặn logic hạn mức mượn sách tối đa

Kịch bản kiểm thử nhằm chứng minh tính đúng đắn của bẫy logic nghiệp vụ tại tầng Service, bảo đảm độc giả không thể vượt qua các ràng buộc định mức đóng gói trong mô hình OOP [5, 13].

Mã số Test Case	TC-02: Đánh chặn giao dịch mượn sách quá hạn mức thẻ
Mục tiêu	Kiểm tra xem hệ thống có kích hoạt bộ lọc bảo vệ và ném ra ngoại lệ BorrowLimitExceededException khi sinh viên mượn quá số lượng quy định hay không [7].
Dữ liệu đầu vào	Tài khoản độc giả sinh viên thường mang mã BD002 (đang giữ 2 cuốn sách trong lịch sử hiện hành), đăng ký mượn thêm gói sách gồm 2 cuốn mới.
Các bước thực hiện	1. Tại giao diện React, chọn chức năng lập Phiếu mượn sách mới [9]. 2. Nhập mã độc giả BD002 và đẩy danh sách 2 mã sách mới vào Form. 3. Nhấn nút "Xác nhận cấp phát phiếu mượn" để đẩy request JSON sang API [4].
Kết quả kỳ vọng	Tầng nghiệp vụ trung tâm (BorrowReturnService) phát hiện tổng số lượng sách sau khi mượn (4 cuốn) vượt quá hằng số đóng gói MAX_BORROW_LIMIT = 3 của thẻ sinh viên thường. Server lập tức dừng luồng xử lý, ném ra BorrowLimitExceededException. Bộ điều hướng GlobalExceptionHandler bắt lỗi và trả về JSON cảnh báo lỗi [10]. Máy trạm React bắt thông điệp và kích hoạt hộp thoại Alert chặn giao dịch [9].
Kết quả thực tế	Server chặn đứng giao dịch, tệp dữ liệu không bị thay đổi, giao diện hiển thị thông báo alert: "Lỗi nghiệp vụ: Độc giả đã vượt quá hạn mức mượn sách tối đa cho phép!".
Đánh giá	ĐẠT (PASSED)

Bảng 5.2: Kịch bản kiểm thử đánh chặn ngoại lệ vượt hạn mức cấp phát

5.1.3 Kịch bản kiểm thử 3: Quyết toán tài chính và Kiểm thử đa hình Strategy Pattern

Kịch bản kiểm thử này ứng dụng kỹ thuật can thiệp mô phỏng dữ liệu (Data Simulation) để kiểm tra tính đúng đắn của thuật toán tính toán ngày trễ hạn và hành vi đa hình động của phân hệ tài chính phạt [8].

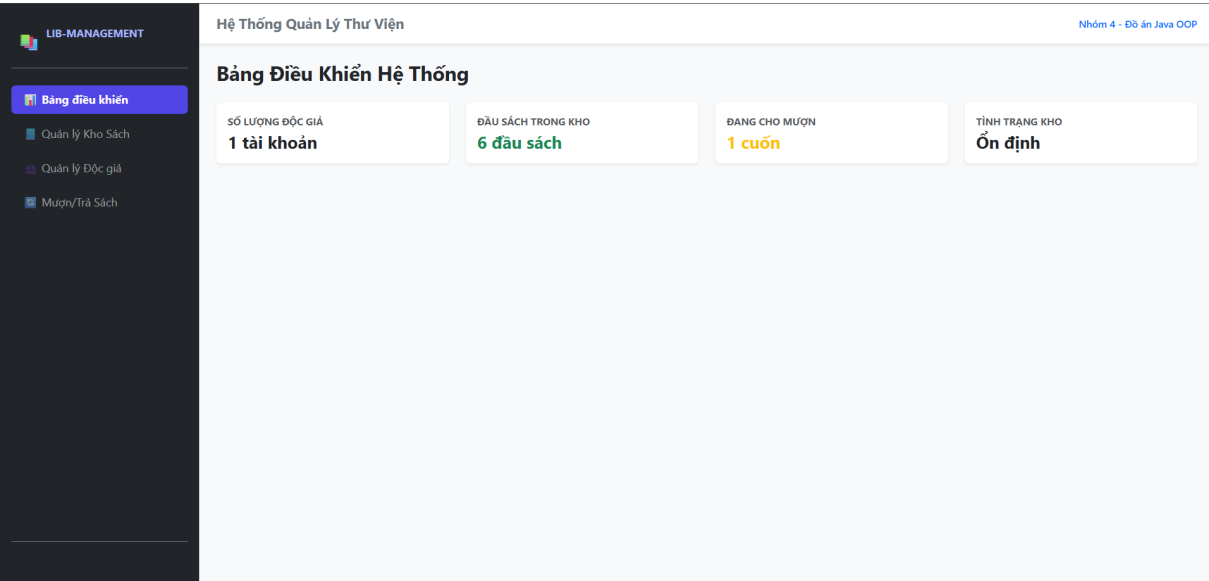
Mã số Test Case	TC-03: Kiểm thử tính toán phí phạt quá hạn thời gian thực
Mục tiêu	Xác thực khả năng đo lường khoảng cách thời gian thông qua ChronoUnit.DAYS và kích hoạt đa hình chiến lược tính phạt tương ứng với từng loại độc giả [13, 8].
Dữ liệu đầu vào	Bản ghi mượn sách đang hoạt động của độc giả Giảng viên (BD005). Tiến hành can thiệp thủ công vào file borrow_tickets.json, điều chỉnh trường dueDate lùi về quá khứ đúng 5 ngày so với mốc LocalDate.now() hiện hành.
Các bước thực hiện	1. Bật bảng điều khiển giám sát lưu thông thời gian thực trên giao diện Dashboard [9]. 2. Định vị đến dòng bản ghi của giảng viên BD005 (cột Hạn hoàn trả hiển thị ngày quá hạn màu đỏ). 3. Nhấp chọn nút tác vụ chức năng "Trả nhanh" trực tiếp trên dòng đó.
Kết quả kỳ vọng	Tầng Backend thu hồi sách an toàn, tính ra số ngày trễ là 5 ngày [13]. Thực thể độc giả gọi hàm đa hình trỏ đến lớp LecturerFinePolicy (định mức phạt 2.000đ/ngày). Hệ thống quyết toán số tiền phạt trả về API là $5 \times 2000 = 10.000$ VND kiểu số nguyên long [5]. Giao diện React hiển thị Pop-up thông báo thu hồi thành công kèm nhãn hiển thị số tiền phạt chuẩn xác [9].
Kết quả thực tế	Hệ thống hiển thị hộp thoại pop-up quyết toán tài chính chính xác số tiền phạt 10.000 VND. Kho sách được cộng trả tự động, bảng theo dõi lập tức xóa dòng bản ghi đó đi.
Đánh giá	ĐẠT (PASSED)

Bảng 5.3: Kịch bản kiểm thử luồng trả sách quá hạn và tính phạt đa hình

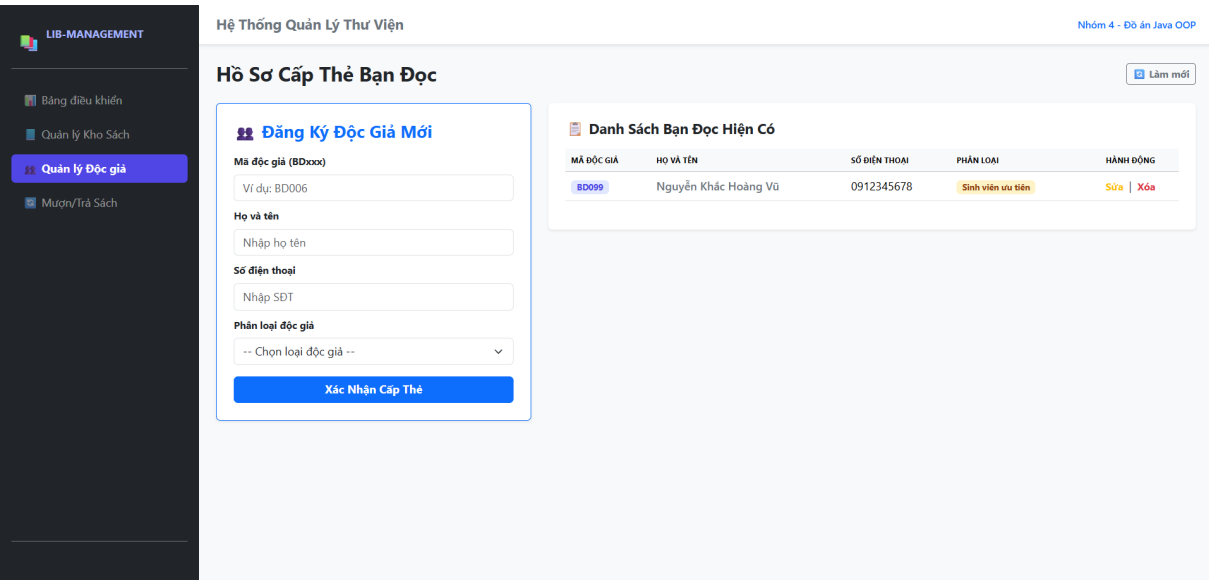
5.2 Ảnh chụp minh chứng giao diện hệ thống (Screenshots Walkthrough)

Để minh chứng cho kết quả kiểm thử thực tế và cấu trúc trực quan hóa giao diện người dùng của đề án, phần này trình bày chuỗi hình ảnh vận hành hệ thống thu thập trực tiếp trong quá trình nghiệm thu máy trạm Single Page Application giao tiếp bất đồng bộ với Spring Boot API [3, 9]:

5.2.1 Giao diện Bảng điều khiển tổng quan (Dashboard)



Hình 5.1: Giao diện Bảng điều khiển tổng quan (Dashboard) tích hợp các cổng số liệu thống kê



Hình 5.2: Giao diện phân hệ Quản lý hồ sơ và phân cấp nhóm Độc giả

LIB-MANAGEMENT

Bảng điều khiển

Quản lý Kho Sách

Quản lý Độc giả

Mượn/Trả Sách

Hệ Thống Quản Lý Thư Viện

Nhóm 4 - Đồ án Java OOP

Quản Lý Danh Mục Kho Sách

Nhập Kho Sách Mới

Mã đầu sách (Bxxx)

Ví dụ: B001

Tên cuốn sách

Nhập tên sách

Tác giả

Nhập tên tác giả

Thể loại

Ví dụ: Giáo trình, Công nghệ

Số lượng tồn

Giá tiền (VND)

10

Xác Nhận Lưu Kho

Danh Sách Sách Trong Tập JSON

MÃ SÁCH	TÊN CUỐN SÁCH	TÁC GIẢ / THỂ LOẠI	SỐ LƯỢNG	ĐƠN GIÁ	HÀNH ĐỘNG
B001	Lập trình Java căn bản	Nguyễn Văn A Giáo trình	15 cuốn	50.000 đ	Sửa Xóa
B002	Cấu trúc dữ liệu và Giải thuật	Trần Văn B Giáo trình	7 cuốn	60.000 đ	Sửa Xóa
B003	Thiết kế hướng đối tượng OOP	Lê Thị C Tham khảo	11 cuốn	75.000 đ	Sửa Xóa
B004	Cơ sở dữ liệu phân tán	Dương Thanh D Giáo trình	5 cuốn	80.000 đ	Sửa Xóa
B005	Phân tích và Thiết kế hệ thống	Hoàng Ngọc E Tham khảo	20 cuốn	90.000 đ	Sửa Xóa
B008	Machine Learning	Trần Đạt Lập trình	5 cuốn	100.000 đ	Sửa Xóa

Hình 5.3: Giao diện phân hệ Quản lý thông tin và số lượng tồn kho Sách

LIB-MANAGEMENT

Bảng điều khiển

Quản lý Kho Sách

Quản lý Độc giả

Mượn/Trả Sách

Hệ Thống Quản Lý Thư Viện

Nhóm 4 - Đồ án Java OOP

Đăng ký cấp phiếu mượn sách

Mã độc giả mượn (BDxxx)

Mã cuốn sách mượn (Bxxx)

Số lượng mượn

1

Phát hành phiếu mượn

Quầy nhận trả sách & Tính phạt trễ hạn

Mã độc giả trả sách (BDxxx)

Ví dụ: BD001

Mã cuốn sách hoàn trả (Bxxx)

Ví dụ: B001

Xác nhận thu hồi sách & Quyết toán

Số Giám Sát Lưu Thông Sách (Mượn Hiện Hành)

MÃ ĐỘC GIẢ	MÃ SÁCH	SỐ LƯỢNG MƯỢN	NGÀY PHÁT HÀNH	HẠN TRẢ SÁCH	HÀNH ĐỘNG NHANH
BD004	B006	3 cuốn	2026-07-05	2026-07-19	Trả nhanh

Hình 5.4: Giao diện cửa sổ Giám sát điều phối mượn trả sách thời gian thực

Hình 5.1 minh chứng cho kiến trúc phẳng hóa dữ liệu giao tiếp mặt trước. Toàn bộ các luồng thông tin phức tạp lồng nhau từ các tệp tin lưu trữ .json đã được Backend bóc tách và phân phối tập trung về màn hình Console giám sát [10]. Thủ thư có cái nhìn toàn cảnh về tổng số lượng đầu sách đang lưu chuyển tại quầy, số lượng thẻ độc giả đang kích hoạt hoạt động và luồng giám sát thời gian thực.

5.2.2 Giao diện Quản lý danh mục Kho sách và Độc giả khôi phục trọn vẹn

Hình 5.2 và Hình 5.3 thể hiện tính chất tương tác mượt mà của biểu mẫu nhập liệu và hệ thống bảng Responsive. Khi thủ thư thực hiện các thao tác CRUD thêm mới, sửa đổi hoặc xóa bỏ đầu sách, các dòng bảng dữ liệu React tự động bắt trạng thái thay đổi bộ nhớ RAM (Reactive rendering) để tái kết xuất giao diện tức thì mà không cần nạp lại trang Web vật lý, tối ưu hóa tối đa trải nghiệm của thủ thư [9].

41

5.2.3 Hộp thoại thông báo tính phí phạt trễ hạn thời gian thực khi trả sách

Hình 5.4 là minh chứng trực quan sinh động nhất cho việc phối hợp giữa tính đa hình của Strategy Pattern dưới tầng Backend Java và cơ chế bẫy lỗi hiển thị thông điệp an toàn tại Frontend [8, 9]. Khi click chọn nút tác vụ "Trả nhanh" trên dòng sổ giám sát của một độc giả quá hạn, một hộp thoại thông báo nổi lập tức xuất hiện, hiển thị minh bạch số tiền quyết toán tài chính phạt trễ hạn được tính toán chuẩn xác từ bộ máy xử lý nghiệp vụ, đảm bảo tính trực quan và khép kín vòng đời giao dịch lưu thông sách.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Kết luận

Sau một thời gian tập trung nghiên cứu nghiêm túc, phân tích bài toán thực tiễn và tiến hành cài đặt mã nguồn phần mềm, nhóm đã hoàn thành trọn vẹn và xuất sắc toàn bộ các mục tiêu, nhiệm vụ đề ra ban đầu của đề án Hệ thống Quản lý Thư viện (Lib-Management). Dự án đã hiện thực hóa thành công một sản phẩm ứng dụng hoàn chỉnh, đạt được các kết quả cốt lõi cả về mặt lý thuyết học thuật hướng đối tượng lẫn kiến trúc thực hành công nghệ kỹ nghệ phần mềm bao gồm [5, 3]:

1. Về mặt Lý thuyết thiết kế hướng đối tượng (OOP): Dự án đã minh chứng sức mạnh tối ưu và tính linh hoạt của tư duy thiết kế hướng đối tượng thông qua việc áp dụng chuẩn mực bốn nguyên lý cốt lõi [13]. Sách chuyên khoa và hồ sơ độc giả được đóng gói an toàn tuyệt đối. Cấu trúc phân cấp hình cây giúp tối ưu mã nguồn kế thừa. Đặc biệt, việc phối hợp nhuần nhuyễn giữa tính đa hình động của ngôn ngữ Java và mẫu thiết kế Chiến lược (Strategy Pattern) cho phân hệ tính phạt tài chính FinePolicy đã giải quyết triệt để bài toán biến động quy tắc nghiệp vụ trong thực tế đời thực, loại bỏ hoàn toàn các cấu trúc mã "bốc mùi" cồng kềnh [6, 8].
2. Về mặt Kiến trúc và Công nghệ hệ thống: Việc dũng cảm lựa chọn và triển khai kiến trúc tách biệt hoàn toàn Frontend - Backend (Decoupled Architecture) đã mang lại hiệu quả vận hành vượt trội [3]. Lõi dịch vụ Java Spring Boot REST API vận hành cực kỳ bền vững, an toàn dữ liệu, bẫy lỗi hệ thống tập trung nghiêm ngặt thông qua GlobalExceptionHandler [10]. Song hành với đó, giao diện React.js Single Page Application đã mang lại trải nghiệm quản trị thời gian thực sinh động, mượt mà bất đồng bộ thông qua cơ chế phản hồi Fetch API siêu tốc [9, 15].
3. Về mặt lưu trữ và triển khai: Giải pháp tự thiết kế tầng lưu trữ Persistence Layer gọn nhẹ dựa trên cơ chế ánh xạ đa hình tệp tin cứng định dạng JSON thông qua thư viện Jackson ObjectMapper giúp hệ thống đạt độ di động an toàn tối đa [11]. Phần mềm có khả năng sao lưu, di chuyển và vận hành chạy tức thì trên mọi hạ tầng máy tính nội bộ của quầy thư viện quy mô vừa và nhỏ mà không đòi hỏi các thủ tục cài đặt cấu hình các hệ quản trị cơ sở dữ liệu quan hệ cồng kềnh phức tạp [1].

Hệ thống đã trải qua chuỗi các kịch bản kiểm thử nghiêm ngặt, vận hành ổn định trong môi trường mạng nội bộ, chứng minh tính đúng đắn của giải pháp thiết kế kiến trúc phần mềm sạch [5].

Hướng phát triển

Mặc dù hệ thống đã đáp ứng tốt các yêu cầu nghiệp vụ đặt ra tại quầy lưu thông, nhóm vẫn định hình các mục tiêu mở rộng cấu trúc và phát triển nâng cấp hệ thống phần mềm trong tương lai để hướng tới một sản phẩm thương mại hóa quy mô lớn [2]:

- Tích hợp Hệ quản trị Cơ sở dữ liệu chính quy: Khi quy mô thư viện mở rộng, số lượng đầu sách và tài khoản độc giả phình to lên tới hàng triệu bản ghi, việc lưu trữ ghi đè tệp JSON vật lý sẽ đối mặt với rào cản tốc độ I/O ổ đĩa [11]. Hướng đi tiếp theo là chuyển đổi và tích hợp hệ quản trị cơ sở dữ liệu quan hệ mạnh mẽ như PostgreSQL hoặc cơ sở dữ liệu hướng tài liệu MongoDB để tối ưu hóa truy vấn lập chỉ mục [1].
- Xây dựng Phân hệ Bảo mật và Phân quyền nâng cao: Nhóm sẽ triển khai hạ tầng an ninh mạng vững chắc dựa trên framework Spring Security kết hợp cơ chế chứng thực mã Token trạng thái JWT (JSON Web Token). Giải pháp này giúp thiết lập ranh giới bảo vệ API nghiêm ngặt, phân tách quyền hạn chi tiết giữa Thủ thư (quyền hạn Admin cao cấp điều phối CRUD danh mục và lưu thông) và Độc giả (quyền hạn User thường trực tiếp đăng nhập qua cổng Web để tra cứu kho tài nguyên và theo dõi lịch sử mượn sách cục bộ) [10].
- Phát triển Module Cảnh báo tự động đa kênh: Tích hợp các cổng dịch vụ bên thứ ba (Third-party APIs) như gửi tin nhắn Email tự động hoặc thông báo đẩy Web Push Notifications để tự động nhắc nhở độc giả khi hệ thống quét phát hiện tài khoản sắp đến hạn trả sách, giảm thiểu tối đa tỷ lệ quá hạn giữ tài sản của thư viện.

TÀI LIỆU THAM KHẢO

- [1] I. Sommerville, Software Engineering. Boston, MA: Pearson, 10th ed., 2015.
- [2] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach. New York, NY: McGraw-Hill, 9th ed., 2020.
- [3] T. Cerny, M. J. Donahoo, and M. Trnka, "Contextual analysis of decoupling in web-based applications," in Proceedings of the International Conference on Research in Adaptive and Convergent Systems, pp. 240-245, 2018.
- [4] R. T. Fielding, Architectural Styles and the Design of Network-based Software Architectures. Ph.D. dissertation, University of California, Irvine, 2000.
- [5] R. C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River, NJ: Prentice Hall, 2008.
- [6] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, 1994.
- [7] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design. Upper Saddle River, NJ: Prentice Hall, 2017.
- [8] E. Freeman and E. Robson, Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software. Sebastopol, CA: O'Reilly Media, 2nd ed., 2020.
- [9] Meta Open Source, "React.js documentation and state management lifecycle." <https://react.dev/>, 2026. Accessed: 2026-07-06.
- [10] Spring Framework Team, "Spring boot reference documentation." <https://docs.spring.io/spring-boot/docs/>, 2025. Accessed: 2026-07-06.
- [11] FasterXML, "Jackson data-binding annotations overview & polymorphic deserialization spec." <https://github.com/FasterXML/jackson-docs>, 2025. Accessed: 2026-07-06.
- [12] M. Fowler, "Inversion of control containers and the dependency injection pattern." <https://martinfowler.com/articles/injection.html>, 2004. Accessed: 2026-07-06.
- [13] H. Schildt, Java: The Complete Reference. New York, NY: McGraw-Hill, 12th ed., 2022.
- [14] Mozilla Developer Network (MDN), "Cross-origin resource sharing (CORS) & fetch API standard." <https://developer.mozilla.org/>, 2026. Accessed: 2026-07-06.

- [15] D. Flanagan, JavaScript: The Definitive Guide. Sebastopol, CA: O'Reilly Media, 7th ed., 2020.